

Smart Pilgrim Companion: Cloud-Based Spiritual Travel & Temple Assistance Platform Using AWS

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Bethala Sumana , APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

ADHIMULAM BHARGAV SAI VISWANATH, 23BQ1A4201, VVITU NAMBURU, (Batch -1)

BEZAWADA MEGHANA, 23BQ1A4218, VVITU NAMBURU, (Batch -1)

DERANGULA VISHNUVARDHAN, 23BQ1A4238, VVITU NAMBURU, (Batch -1)

BODEPUDI NAGA SOUMYA, 23BQ1A4222, VVITU NAMBURU, (Batch -1)

JUJJURI LAKSHMI SOWMYA, 23BQ1A4269, VVITU NAMBURU, (Batch -1)

ACKNOWLEDGEMENT

We express our sincere gratitude to Vasireddy Venkatadri Institute of Technology (VVIT), our faculty mentors, and all project coordinators for their guidance and support throughout the completion of this project.

We also thank the APSSDC AWS learning environment and project guidance process for helping us understand cloud deployment, backend hosting, database connectivity, monitoring, and documentation practices.

This project helped us apply practical knowledge in React, Flask, SQL data modeling, REST API development, AWS EC2 deployment, Nginx reverse proxy configuration, Gunicorn service hosting, AWS RDS MySQL connectivity, IAM access control, and CloudWatch monitoring.

We are thankful to every team member for contributing to development, deployment, validation, documentation, and testing. The combined effort resulted in a working cloud-based spiritual travel and temple assistance platform.

Team Members

ADHIMULAM BHARGAV SAI VISWANATH

BEZAWADA MEGHANA

BODEPUDI NAGA SOUMYA

DERANGULA VISHNUVARDHAN

JUJJURI LAKSHMI SOWMYA

ABSTRACT

Smart Pilgrim Companion is a cloud-based spiritual travel and temple assistance platform designed to help pilgrims explore temples, view temple information, plan visits, estimate travel budgets, review travel routes, and receive recommendation-based planning support. The primary objective of the project is to provide a user-friendly and efficient solution that simplifies pilgrimage planning by integrating modern web technologies with cloud computing. The project is implemented using a **React + Vite** frontend and a **Flask** backend. The frontend includes modules such as **Home**, **Temples**, **Temple Details**, **Explore**, **Planner**, and **About**, while the backend exposes **RESTful APIs** for temple information, planner data, recommendations, travel routes, analytics, performance metrics, and health validation. The application stores and manages temple information, travel routes, budgets, schedules, nearby places, and metadata using **SQLAlchemy** models connected to the database. The application supports both local development and cloud deployment. Local execution is performed using **Vite**, **Flask**, and a local database, while cloud deployment utilizes **Amazon Web Services (AWS)** with **Amazon EC2**, **Amazon RDS MySQL**, **Nginx**, **Gunicorn**, **IAM**, and **Amazon CloudWatch** for secure hosting, database management, monitoring, and logging. The backend automatically reads the **DATABASE_URL** environment variable and falls back to the local **SQLite** database when a cloud database connection is unavailable. The project includes deployment screenshots, testing evidence, GitHub deployment validation, AWS hosting proof, team contribution records, and final output screenshots. The successful implementation of the Smart Pilgrim Companion demonstrates practical knowledge of **full-stack web development**, **cloud migration**, **AWS deployment**, and **project documentation**, providing a scalable foundation for future enhancements and academic project submission

Table of Contents

1. INTRODUCTION.....	1
1.1 Project Overview.....	1
1.2 Temples Covered.....	1
1.3 Need for the Project.....	1
1.4 Project Features.....	2
1.5 User Journey Diagram.....	2
1.6 Screenshot Evidence.....	3
2. Problem Statement.....	4
2.1 Problem Statement.....	4
2.2 Objectives.....	4
2.3 Existing Challenges.....	4
2.4 Proposed Solution.....	5
2.5 Scope.....	5
2.6 Problem-to-Solution Flow.....	5
3.METHODOLOGY.....	6
3.1 Development Methodology.....	6
3.2 Project Workflow.....	6
3.3 Repository-Based Analysis.....	7
3.4 Data Methodology.....	7
3.5 API Methodology.....	8
4. TECHNOLOGY STACK.....	9
4.1 Overview.....	9
4.2 Frontend Stack.....	9
4.3 AWS Services.....	9
4.4 Database Stack.....	10
4.5 AWS Stack.....	10
4.6 Tooling and Configuration.....	10
4.7 Stack Diagram.....	11
5.SYSTEM DESIGN / ARCHITECTURE.....	12
5.1 System Architecture.....	12
5.2 Frontend Design.....	12
5.3 Backend Design.....	12
5.4 Module Flow.....	13
5.5 Request Lifecycle.....	13
5.6 Database Relation Diagram.....	14
5.7 AWS Network Flow.....	14
5.8 CI/CD Flow.....	15
5.9 Design Evidence.....	15
6. IMPLEMENTATION.....	17
6.1 Repository Implementation.....	17
6.2 Frontend Implementation.....	17
6.3 Frontend API Integration.....	18
6.4 Backend Implementation.....	18

6.5 Planner Implementation.....	18
6.6 Recommendation Implementation.....	19
6.7 Database Implementation.....	19
6.8 API Response Structure.....	19
6.9 Implementation Flow Diagram.....	20
6.10 UI Evidence.....	21
7. AWS Deployment.....	23
7.1 Deployment Overview.....	23
7.2 Localhost Deployment.....	23
7.3 GitHub Deployment.....	23
7.4 AWS Migration Flow.....	24
7.5 AWS Runtime Architecture.....	25
7.6 Backend Environment Configuration.....	26
7.7 Nginx and Gunicorn Role.....	26
7.8 Cloud Monitoring.....	27
7.9 AWS Deployment Evidence.....	27
8: Testing and Results.....	33
8.1 Testing Overview.....	33
8.2 Functional Testing.....	33
8.3 API Testing.....	33
8.4 Deployment Validation.....	33
8.5 Cloud Validation.....	34
8.6 Testing Flow.....	34
8.7 Results.....	35
8.8 Testing Evidence Screenshots.....	35
8.9 Final Output Screenshots.....	40
Chapter 9: Team Contributions.....	45
9.1 Team Contribution Overview.....	45
9.2 Contribution Mapping.....	45
9.3 Responsibility Flow.....	45
9.4 Collaboration Evidence.....	46
Chapter 10: Conclusion.....	48
Chapter 11: Future Scope.....	49
11.1 Overview.....	49
11.2 Future Enhancements.....	49
11.3 Cloud Enhancement Scope.....	49
11.4 Future Scope Diagram.....	49
References.....	50
Repository References.....	50
Technology References.....	50
Dataset References.....	50
Evidence References.....	51

List of Figures

S.No	Name	PageNo
1	User Journey Diagram	2
2	GitHub Pages home page	3
3	Temples page showing available pilgrimage destinations	3
4	Problem-to-Solution Flow	5
5	Project Workflow	6
6	API Methodology	8
7	Evidence Collection	8
8	Stack Diagram	11
9	System Architecture	12
10	Module Flow	13
11	Request Lifecycle	13
12	Database Relation Diagram	14
13	AWS Network Flow	14
14	CI/CD Flow	15
15	Amazon RDS configuration for production database setup	15
16	Amazon EC2 security group for AWS network configuration	15
17	Implementation Flow	20
18	Explore page in GitHub deployment	21
19	About page in GitHub deployment	21
20	Explore page after AWS deployment	21

21	Planner page after AWS deployment	21
22	GitHub Deployment Workflow	23
23	AWS Migration Workflow	24
24	AWS Runtime Architecture	25
25	Nginx & Gunicorn Role	26
26	AWS deployment home page	27
27	AWS deployment home page in dark mode	27
28	AWS deployment templates page	27
29	AWS deployment planner page	27
30	AWS deployment about page	27
31	Nginx service status on Amazon	27
32	Gunicorn service status on Amazon EC2	27
33	Amazon EC2 monitoring dashboard	27
34	Amazon RDS MySQL configuration	27
35	Amazon RDS monitoring dashboard	27
36	Amazon RDS security group configuration	27
37	Amazon EC2 security group configuration	27
38	Amazon S3 frontend ZIP file upload	27
39	Cloud Validation	34
40	Testing Flow	34
41	Testing evidence output 1	35
42	Testing evidence output 2	35

43	Testing evidence output 3	35
44	Testing evidence output 4	35
45	Testing evidence output 5	35
46	Testing evidence output 6	35
47	Testing evidence output 7	35
48	Testing evidence output 8	35
49	Testing evidence output 10	35
50	Final output screen 1	40
51	Final output screen 2	40
52	Final output screen 3	40
53	Final output screen 4	40
54	Final output screen 5	40
55	Final output screen 6	40
56	Final output screen 7	40
57	Final output screen 8	40
58	Final output screen 9	40
59	Responsibility Flow	45
60	GitHub organization homepage 1	46
61	GitHub repository evidenc 1	46
62	GitHub branch collaboration e 1	46
63	Team members evidence 1	46
64	Future Scope Diagram	49

List of Tables

S.No	Name	PageNo
1	Team Members	1
2	Frontend Stack	9
3	AWS Services	9
4	Database Stack	10
5	AWS Stack	10
6	Contribution Mapping	45

1. INTRODUCTION

1.1 Project Overview

Smart Pilgrim Companion is a web-based spiritual travel and temple assistance platform designed to simplify pilgrimage planning and enhance the overall travel experience for devotees. The application enables users to explore temples, access detailed temple information, discover nearby attractions, compare travel routes, estimate travel budgets, and receive recommendation-based travel planning support through a single integrated platform.

The project is implemented as a **full-stack web application**, consisting of a **React + Vite** frontend and a **Flask** backend that communicate through **RESTful APIs**. Temple information, travel routes, budgets, schedules, nearby places, and recommendation data are managed using structured datasets and **SQLAlchemy** models connected to a database. The project also includes cloud deployment using **Amazon Web Services (AWS)**, demonstrating modern web application development and cloud migration practices.

1.2 Temples Covered

The Smart Pilgrim Companion currently focuses on three major pilgrimage destinations in **Andhra Pradesh**, providing comprehensive information and travel assistance for each temple.

- **Tirumala**
- **Srisailem**
- **Srikalahasti**

The platform stores information related to temple history, travel routes, estimated budgets, nearby attractions, schedules, and user planning scenarios, allowing pilgrims to make informed travel decisions.

1.3 Need for the Project

Planning a pilgrimage often requires collecting information from multiple websites and sources, making the process time-consuming and inconvenient. Pilgrims typically need temple details, transportation options, accommodation information, travel costs, nearby attractions, and suggested itineraries before beginning their journey.

The **Smart Pilgrim Companion** addresses these challenges by providing a single platform that combines temple information, travel planning, recommendation services, and cloud-based deployment. The project also demonstrates the migration of a full-stack application from a local development environment to **GitHub Pages** and **AWS Cloud**, highlighting practical cloud computing concepts.

1.4 Project Features

The major features of the Smart Pilgrim Companion include:

- **Home** page providing an overview of the platform.
- **Temples** page for browsing available pilgrimage destinations.
- **Temple Details** page containing historical and travel information.
- **Explore** page for discovering temples and nearby attractions.
- **Planner** page for generating travel plans based on user preferences.
- **Recommendation** system for route and budget planning.
- **RESTful APIs** for temples, planner, routes, recommendations, analytics, and health monitoring.
- **Cloud deployment** using GitHub Pages and Amazon Web Services (AWS).

1.5 User Journey Diagram

The following diagram illustrates the overall workflow of the Smart Pilgrim Companion from

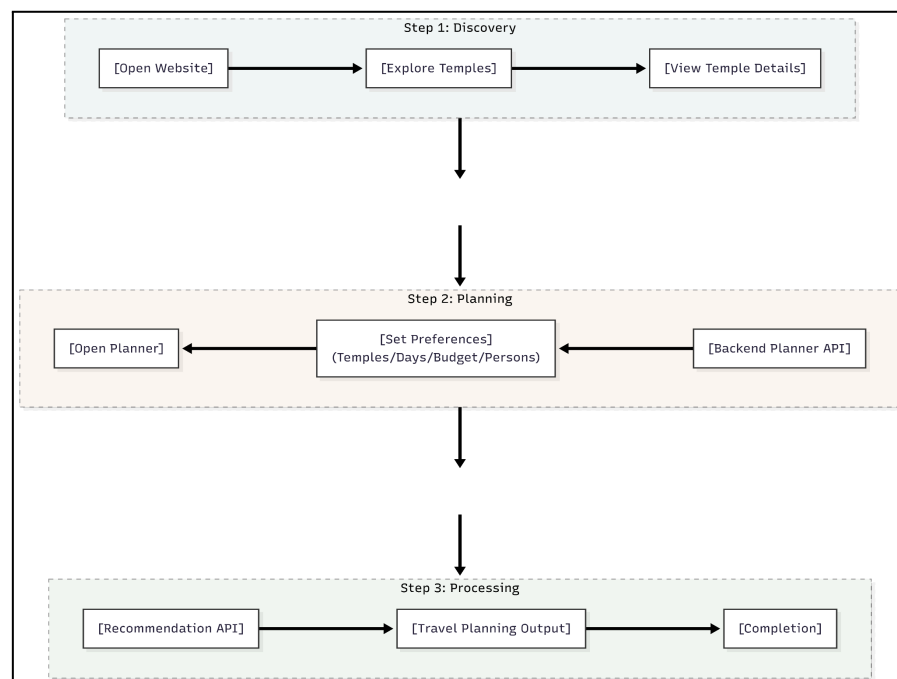


Figure 1: User Journey Diagram

1.6 Screenshot Evidence

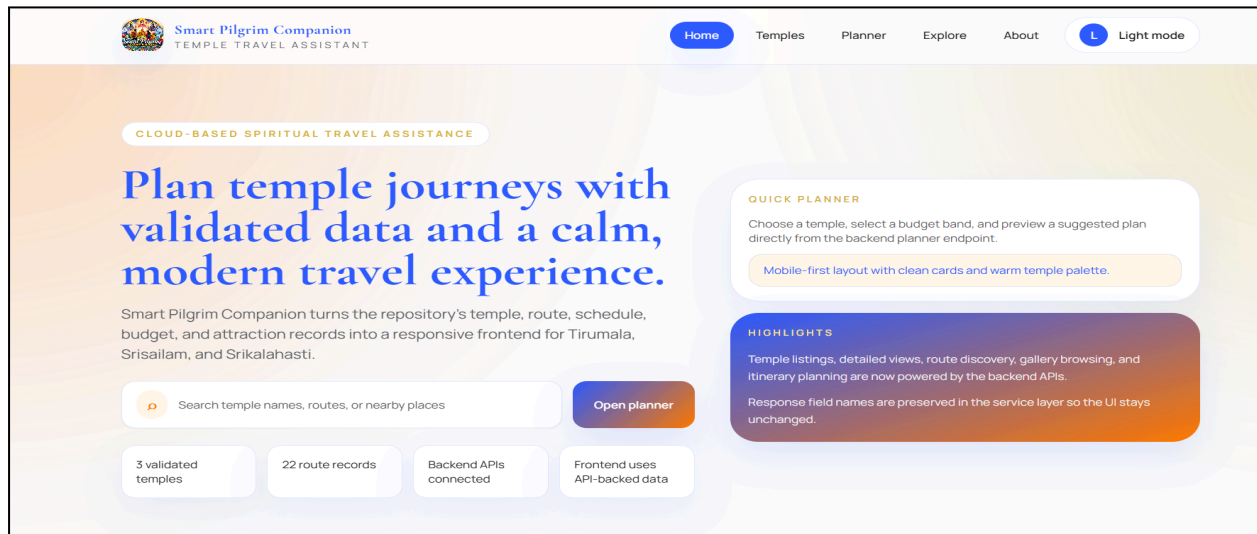


Figure 2: GitHub Pages home page showing

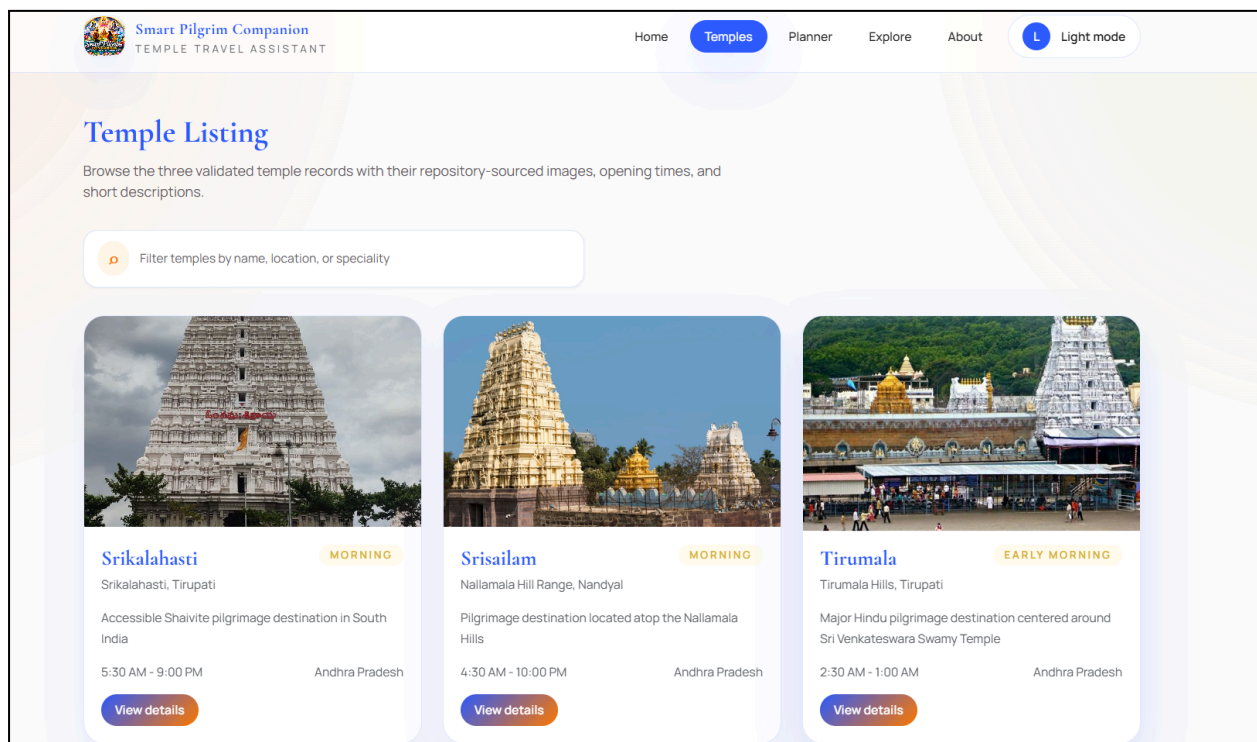


Figure 3: Temples page showing available pilgrimage destinations

2. Problem Statement

2.1 Problem Statement

Pilgrims require reliable temple information, travel routes, schedules, nearby places, and budget planning support before beginning a spiritual journey. In many cases, these details are scattered across different websites, PDF references, personal notes, and local knowledge sources. This makes planning difficult, especially for users who want a simple interface that combines spiritual destination information with practical travel assistance.

The problem addressed by this project is the design and deployment of a cloud-based full-stack platform that provides temple exploration, route planning, budget estimation, and recommendation support using structured datasets and AWS deployment services.

2.2 Objectives

- Build a React + Vite frontend for temple exploration and travel planning.
- Build a Flask backend with JSON APIs for temples, planner, routes, recommendations, analytics, performance, and health.
- Use repository datasets as the source of truth for temple, route, budget, schedule, nearby place, metadata, and scenario information.
- Provide local development support through Vite and Flask.
- Deploy the frontend through GitHub Pages.
- Migrate the production architecture to AWS EC2, Nginx, Gunicorn, Flask, and AWS RDS MySQL.
- Use IAM for access control and CloudWatch for monitoring evidence.
- Prepare academic documentation with Mermaid diagrams and screenshot placeholders.

2.3 Existing Challenges

- Temple planning data is distributed across different sources.
- Users need route, schedule, budget, and nearby place information together.
- Static frontend-only information is not enough for planner and recommendation workflows.
- Production deployment requires separation of frontend hosting, backend service execution, reverse proxy configuration, and database connectivity.
- Cloud validation requires monitoring and evidence through AWS services.

2.4 Proposed Solution

The proposed system is a full-stack cloud-ready application:

- The frontend provides user-facing pages and calls backend APIs through `frontend/src/services/api.js`.
- The backend serves structured JSON responses through Flask routes.
- The data layer uses SQLAlchemy models mapped to temple, budget, route, schedule, and nearby place records.
- Local development uses SQLite through `database/smart_pilgrim.db`.
- AWS deployment uses environment-driven database configuration for AWS RDS MySQL.
- EC2 hosts the application backend with Gunicorn and Nginx.
- CloudWatch and AWS screenshots provide monitoring and validation evidence.

2.5 Scope

The project scope includes temple browsing, temple details, planner workflow, route and budget support, recommendation generation, API validation, deployment evidence, and academic documentation. The system is not a payment platform, booking platform, or live government darshan integration service.

2.6 Problem-to-Solution Flow

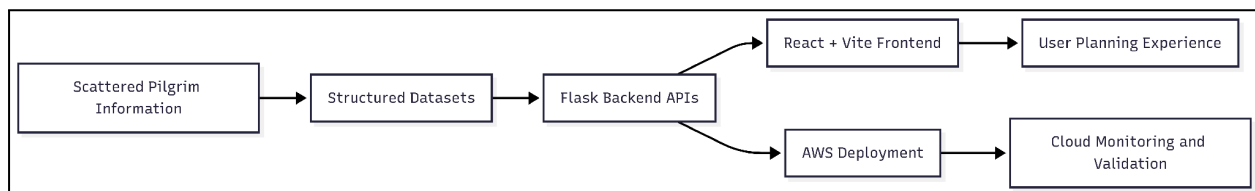


Figure 4: Problem-to-Solution Flow

3.METHODOLOGY

3.1 Development Methodology

The project followed an implementation-first methodology. The team developed the frontend, backend, data layer, and deployment setup in stages. Each stage produced working outputs and evidence screenshots.

The repository status indicates feature freeze, allowing only deployment configuration, bug fixes, and production documentation after the checkpoint.

3.2 Project Workflow

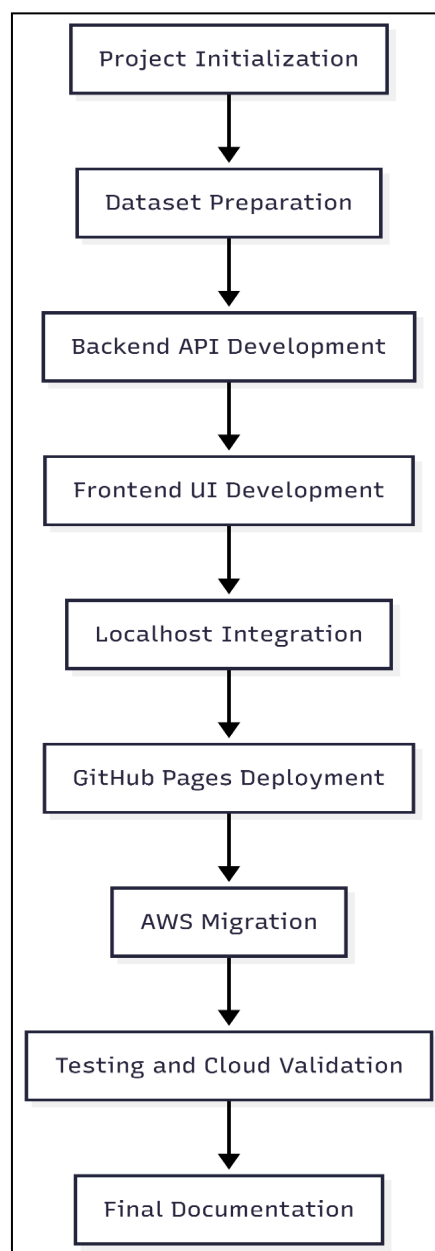


Figure 5: Project Workflow

3.3 Repository-Based Analysis

The documentation was prepared after analyzing:

- README.md
- frontend/package.json
- frontend/src/App.jsx
- frontend/src/services/api.js
- frontend/src/services/templeService.js
- frontend/src/services/plannerService.js
- frontend/DEPLOYMENT_GUIDE.md
- backend/README.md
- backend/app.py
- backend/config.py
- backend/routes/temple_routes.py
- backend/services/planner_service.py
- backend/services/recommendation_engine.py
- database/schema.sql
- database/DATA_DICTIONARY.md
- deployment/aws/backend-audit.md
- deployment/aws/frontend-audit.md
- deployment/ProjectProofs/

3.4 Data Methodology

The backend README states that existing CSV and JSON files are the source of truth. The data files include:

- data/temples.csv
- data/travel_routes.csv
- data/budgets.csv
- data/schedules.csv
- data/temple_places.csv
- data/metadata.json
- data/user_scenarios.json

The seed script database/seed.py creates and populates database/smart_pilgrim.db for local development. For AWS deployment, backend/config.py reads DATABASE_URL.

3.5 API Methodology

The backend exposes REST endpoints under /api. The frontend service layer calls these endpoints and normalizes the responses before rendering them in pages.

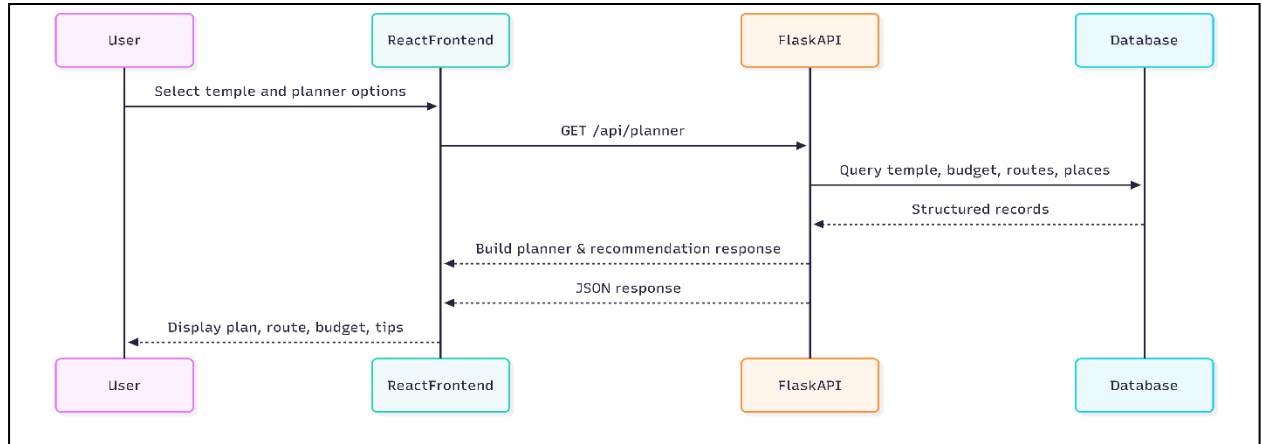


Figure 6: API Methodology

3.6 Deployment Methodology

Deployment was handled in three stages:

- Localhost validation using Vite and Flask.
- GitHub Pages frontend deployment using gh-pages.
- AWS migration using EC2, Nginx, Gunicorn, Flask, RDS MySQL, IAM.

3.7 Evidence Collection

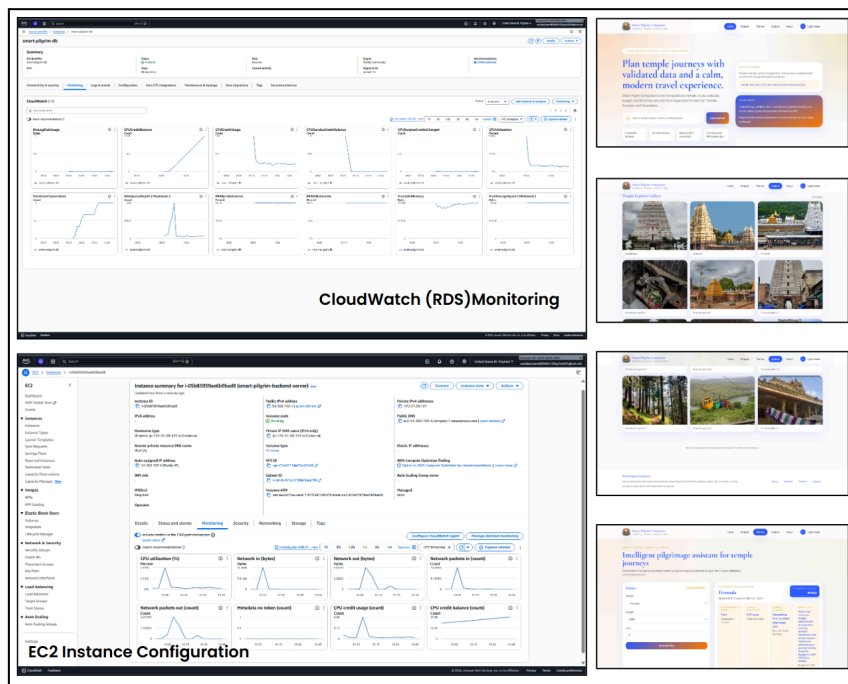


Figure 7: Evidence Collection

4. TECHNOLOGY STACK

4.1 Overview

The Smart Pilgrim Companion project uses a full-stack web architecture with a React frontend, Flask backend, SQLAlchemy data layer, and AWS deployment components.

4.2 Frontend Stack

Technology	Actual Usage
React	User interface pages and reusable components
Vite	Frontend development server and production build
React Router DOM	Routes for Home, Temples, Temple Details, Planner, Explore, About
Axios	HTTP API requests
Tailwind CSS	Utility-based UI styling
gh-pages	GitHub Pages deployment

Table 1: Frontend Stack

4.3 AWS Services

Technology	Actual Usage
Python	Backend language
Flask	REST API framework
Flask-CORS	Cross-origin frontend/backend API access
Flask-SQLAlchemy	ORM layer
pandas	Dataset handling support

Technology	Actual Usage
Gunicorn	Production WSGI server for AWS deployment

Table 2: AWS Services

4.4 Database Stack

Environment	Database
Local development	SQLite database at database/smart_pilgrim.db
AWS deployment target	AWS RDS MySQL using DATABASE_URL

Table 3: Database Stack

4.5 AWS Stack

AWS Component	Project Usage
EC2	Hosts backend/application deployment
Nginx	Reverse proxy for backend routing
Gunicorn	Runs Flask application as production service
RDS MySQL	Production relational database target
IAM	Access control and permission management
CloudWatch	Monitoring evidence for EC2 and application behavior
S3	Evidence includes frontend zip storage screenshot

Table 4: AWS Stack

4.6 Tooling and Configuration

- frontend/package.json defines dev, build, preview, predeploy, and deploy.
- frontend/src/services/api.js reads VITE_API_URL and falls back to localhost.
- backend/config.py reads DATABASE_URL and SECRET_KEY.
- backend/requirements.txt includes Flask, Flask-CORS, Flask-SQLAlchemy, pandas, and gunicorn.

4.7 Stack Diagram

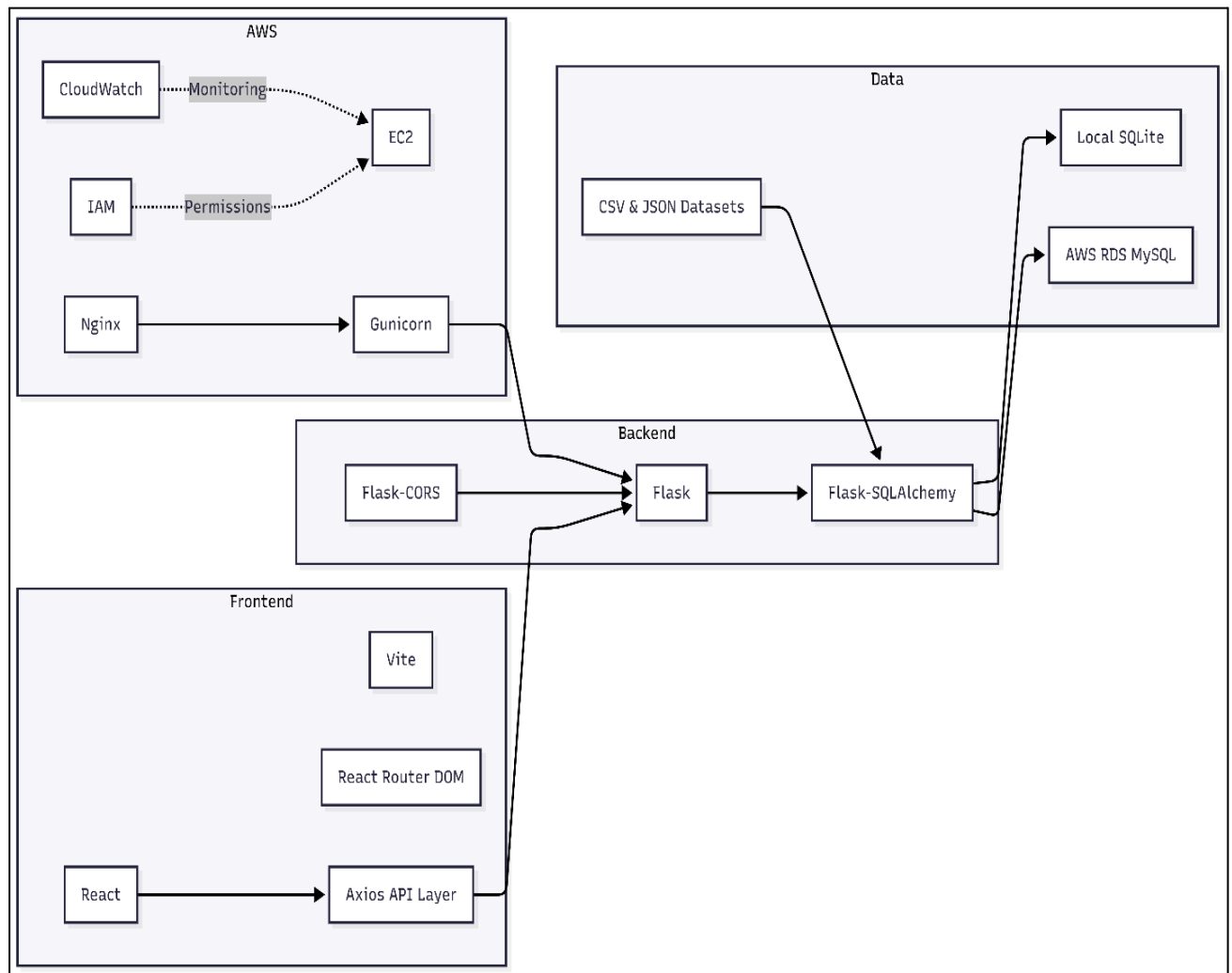


Figure 8: Stack Diagram

5.SYSTEM DESIGN / ARCHITECTURE

5.1 System Architecture

Smart Pilgrim Companion follows a client-server architecture. The frontend is built using React + Vite and communicates with the Flask backend through REST APIs. The backend uses SQLAlchemy models and database configuration that supports local SQLite and AWS RDS MySQL.

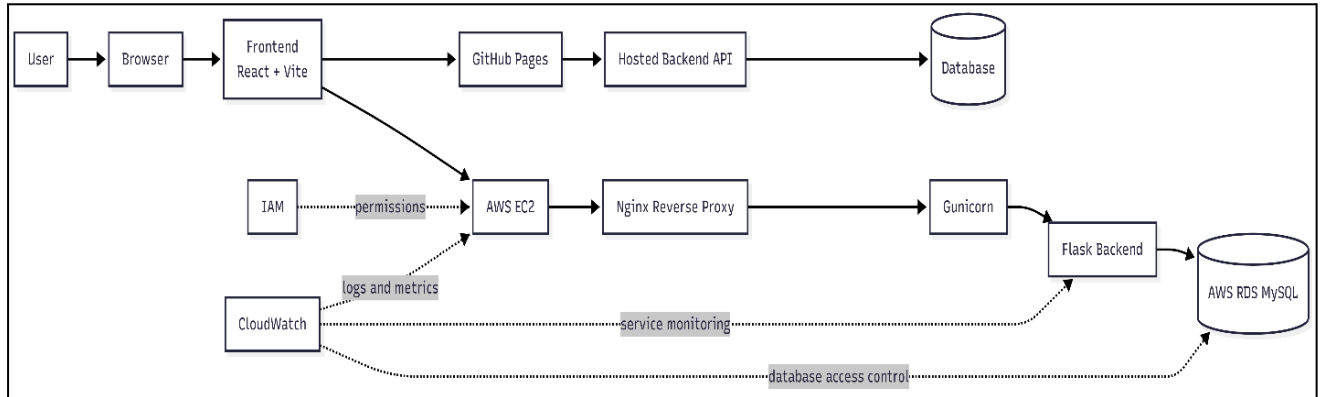


Figure 9: System Architecture

5.2 Frontend Design

The frontend is organized into pages, components, services, data, styles, and static assets.

Important frontend routes from frontend/src/App.jsx:

- / -> Home page
- /temples -> Temples listing
- /temples/:templeId -> Temple details
- /planner -> Travel planner
- /explore -> Explore page
- /about -> About page

The API base configuration is handled in frontend/src/services/api.js. It reads VITE_API_URL and falls back to <http://localhost:5000>.

5.3 Backend Design

The backend is organized into:

- backend/app.py - Flask app factory and health route.
- backend/config.py - database and environment configuration.
- backend/routes/temple_routes.py - API endpoints.
- backend/services/ - business logic for temples, search, planner, recommendations,

analytics.

- backend/models/ - SQLAlchemy models.
- backend/utlis/ - response and validation helpers.

5.4 Module Flow

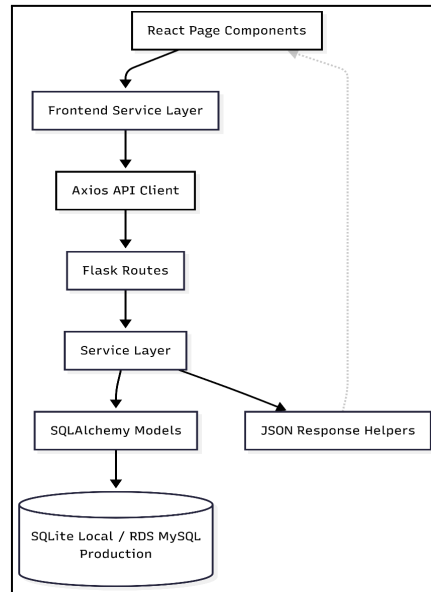


Figure 10: Module Flow

5.5 Request Lifecycle

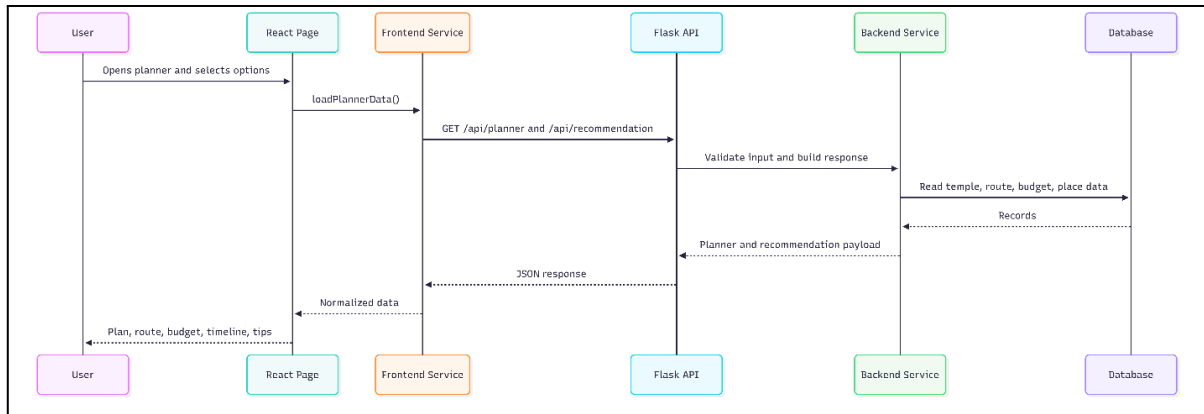


Figure 11: Request Lifecycle

5.6 Database Relation Diagram

The local schema is defined in database/schema.sql. The same logical model supports migration to AWS RDS MySQL.

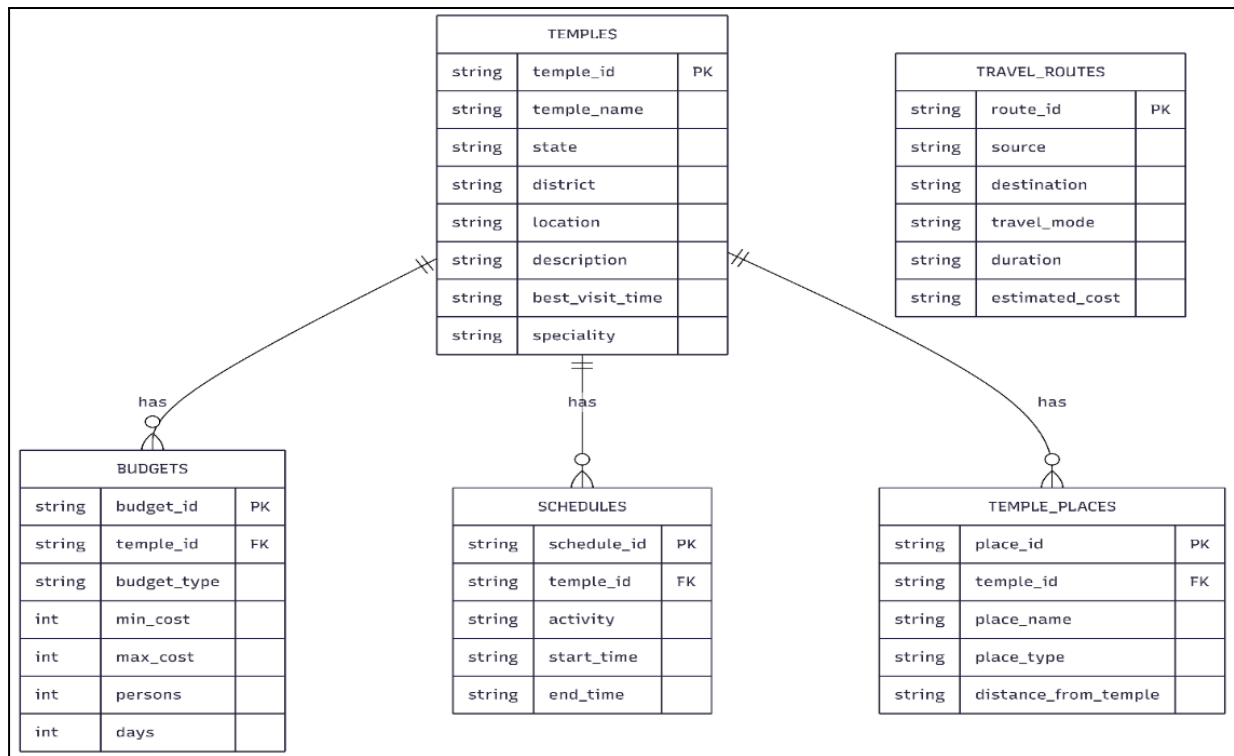


Figure 12: Database Relation Diagram

5.7 AWS Network Flow

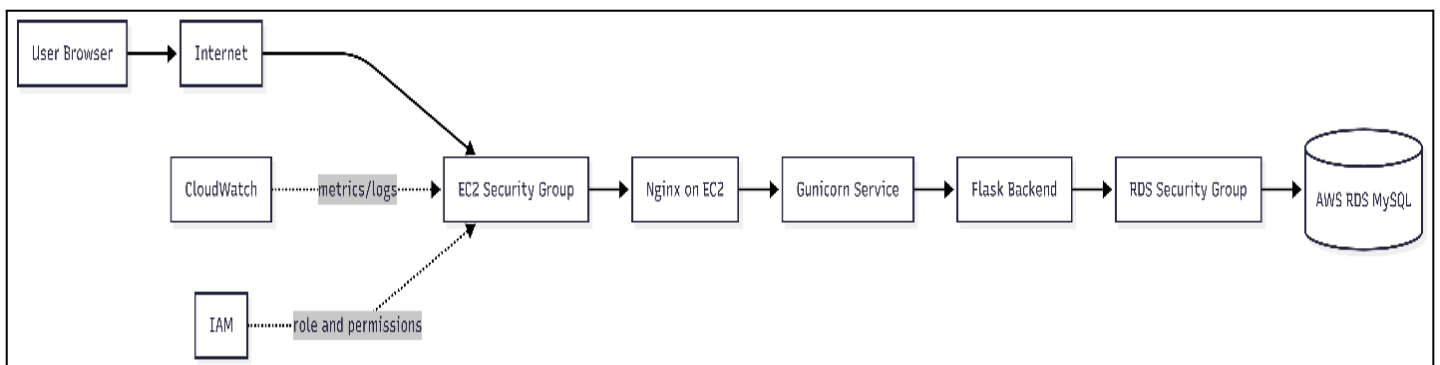


Figure 13: AWS Network Flow

5.8 CI/CD Flow

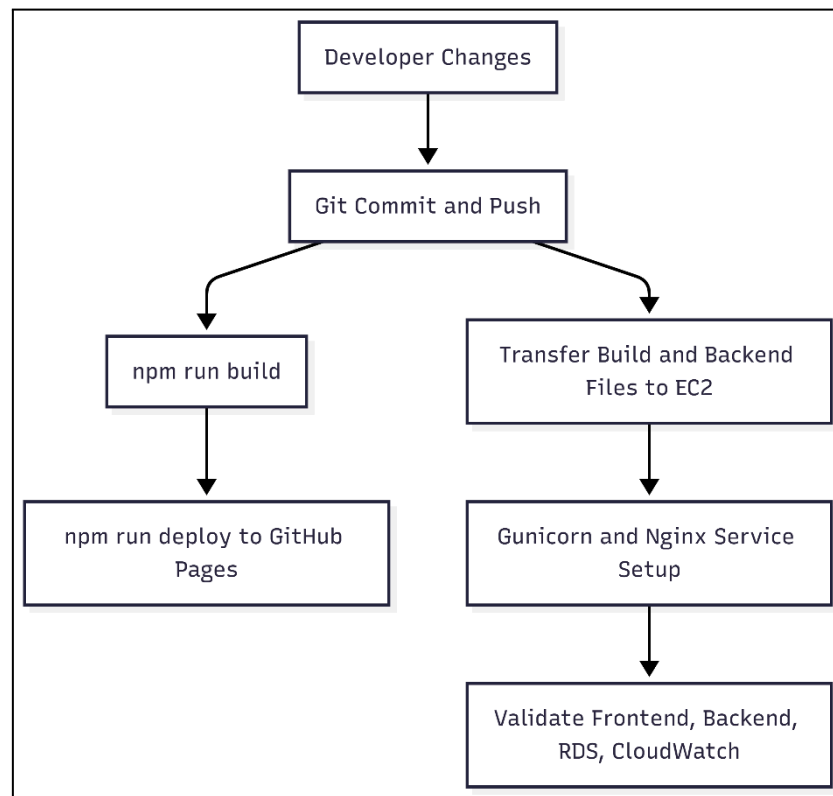


Figure 14: CI/CD Flow

5.9 Design Evidence

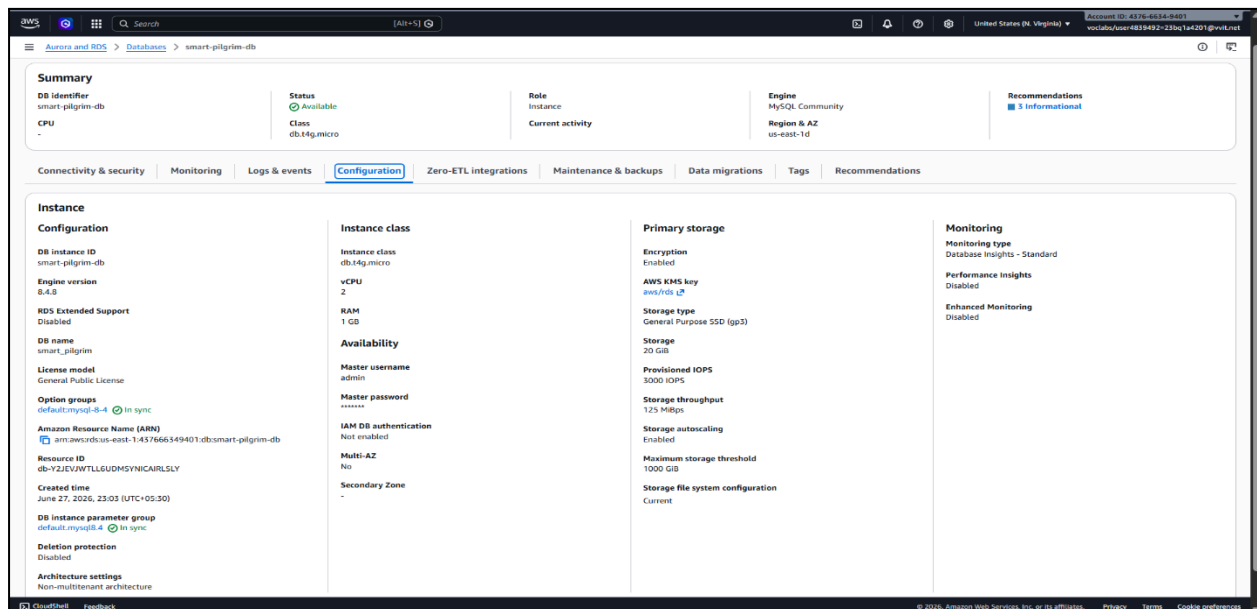


Figure 15: Amazon RDS configuration for production database setup

Search

[Alt+S]

United States (N. Virginia)

Account ID: 4376-6634-9401

voclabs/user4839492=23bq1a4201@vvvit.net

EC2

Security Groups

sg-083480e103e6c8043 - launch-wizard-4

sg-083480e103e6c8043 - launch-wizard-4

Actions

Details

Security group name

launch-wizard-4

Security group ID

sg-083480e103e6c8043

Description

launch-wizard-4 created 2026-06-27T18:47:59.932Z

VPC ID

vpc-07e4311da05dd74a9

Owner

437666349401

Inbound rules count

3 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (3)

Manage tags

Edit inbound rules

Search

Name

Security group rule ID

IP version

Type

Protocol

Port range

Source

Description

-

sgr-0486bb9ac95c8d55a

IPv4

SSH

TCP

22

0.0.0.0/0

-

-

sgr-0e14af8c9b5232df3

IPv4

HTTP

TCP

80

0.0.0.0/0

-

-

sgr-0c19a88b7a1cdf81

IPv4

HTTPS

TCP

443

0.0.0.0/0

-

CloudShell

Feedback

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Figure 16: Amazon EC2 security group for AWS network configuration

16

6. IMPLEMENTATION

6.1 Repository Implementation

The actual repository is divided into frontend, backend, database, data, deployment, and documentation areas.

```
smart-pilgrim-companion/  
  backend/  
  data/  
  database/  
  deployment/  
  docs/  
  frontend/  
  README.md  
  RELEASE_NOTES.md
```

6.2 Frontend Implementation

The frontend is implemented with React + Vite. The important files include:

- frontend/src/main.jsx
- frontend/src/App.jsx
- frontend/src/pages/HomePage.jsx
- frontend/src/pages/TemplesPage.jsx
- frontend/src/pages/TempleDetailsPage.jsx
- frontend/src/pages/TravelPlannerPage.jsx
- frontend/src/pages/ExplorePage.jsx
- frontend/src/pages/AboutPage.jsx
- frontend/src/components/
- frontend/src/services/
- frontend/src/data/

The frontend service layer uses axios and reads the backend base URL

from VITE_API_URL.

6.3 Frontend API Integration

frontend/src/services/api.js defines:

- DEFAULT_API_URL = http://localhost:5000
- API_BASE_URL = <VITE_API_URL or localhost>/api
- request interceptors for loading state
- response error normalization
- GET request de-duplication
- query string helper

The application service files use this API client:

- templeService.js loads temples, temple details, and search results.
- plannerService.js loads planner and recommendation data.
- routeService.js loads route data.
- budgetService.js loads budget information.

6.4 Backend Implementation

The backend is implemented with Flask. backend/app.py creates the Flask app, loads environment variables, applies configuration, initializes SQLAlchemy, enables CORS, registers the temple blueprint under /api, and provides a root health response.

The backend route file backend/routes/temple_routes.py includes:

- GET /api/health
- GET /api/temples
- GET /api/temples/<identifier>
- GET /api/routes
- GET /api/planner
- GET /api/recommendation
- GET /api/performance
- GET /api/analytics/summar

6.5 Planner Implementation

backend/services/planner_service.py resolves the selected temple, normalizes budget type, selects budgets and routes, builds itinerary steps, increments analytics metrics, and returns a structured planning payload.

Planner output includes:

- temple details
- selected route
- budget options
- route options
- timeline
- nearby places
- smart tips
- risk notes
- estimated budget
- AI-style recommendation node

6.6 Recommendation Implementation

backend/services/recommendation_engine.py builds route and budget recommendations using the available database records. It calculates route scoring using:

- budget fit
- travel time score
- temple priority score
- nearby places score

The response includes recommended route, estimated budget, best visit time, smart tip, confidence score, route details, and budget details.

6.7 Database Implementation

The database schema includes:

- temples
- travel_routes
- budgets
- schedules
- temple_places

The local database is generated as database/smart_pilgrim.db using database/seed.py. The project data is stored under data/ using CSV and JSON files.

6.8 API Response Structure

The backend returns JSON responses in a consistent pattern:


```

{
  "status": "success",
  "message": "temples fetched",
  "data": []
}

```

Validation errors return structured error responses with useful messages, especially in planner validation for missing temple, invalid days, and invalid budget selection.

6.9 Implementation Flow Diagram

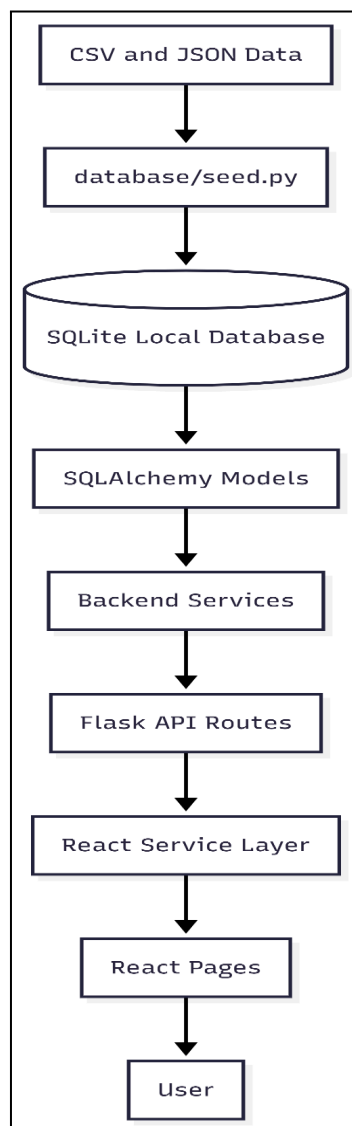


Figure 17: Implementation Flow

6.10 UI Evidence

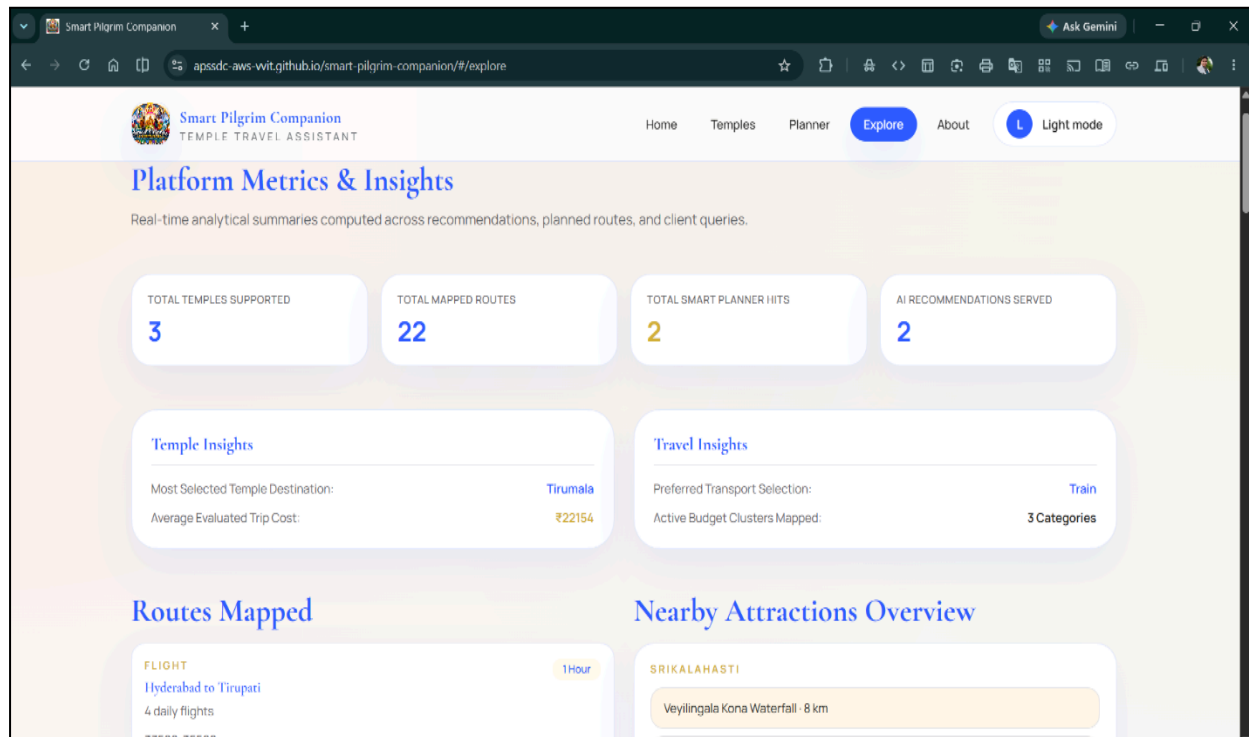


Figure 18: Explore page in GitHub deployment

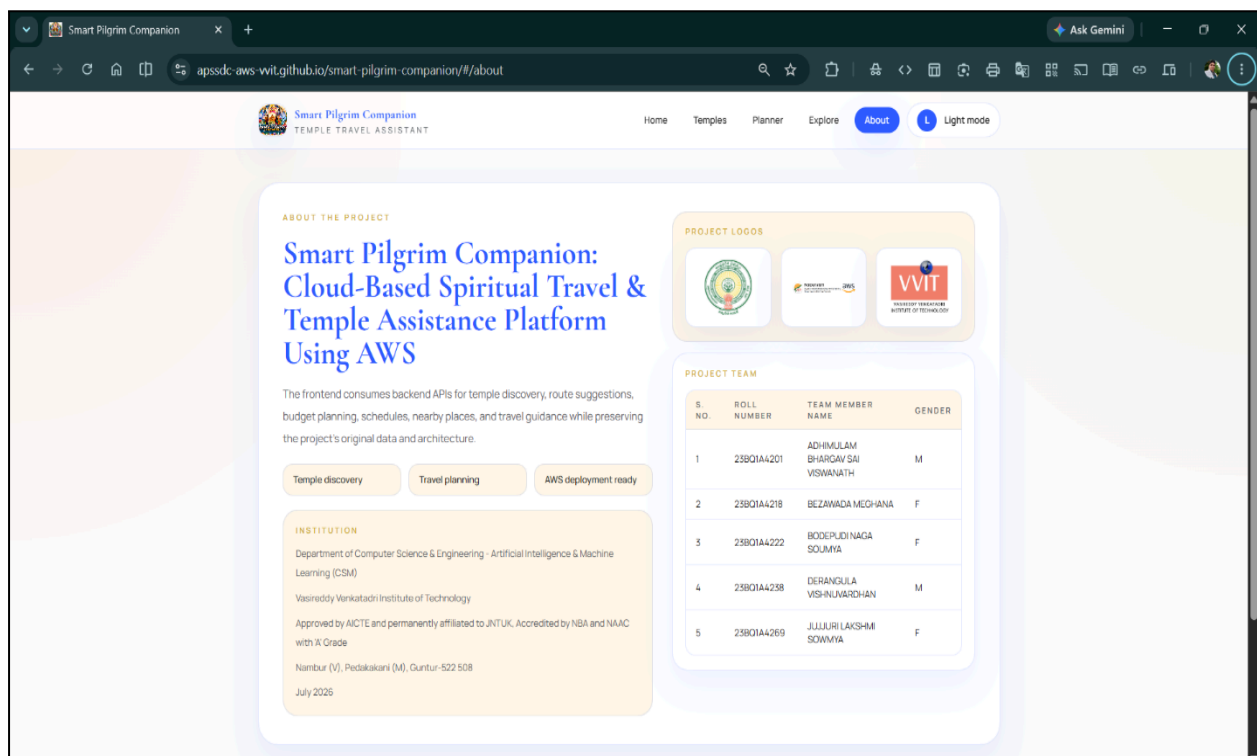


Figure 19: About page in GitHub deployment

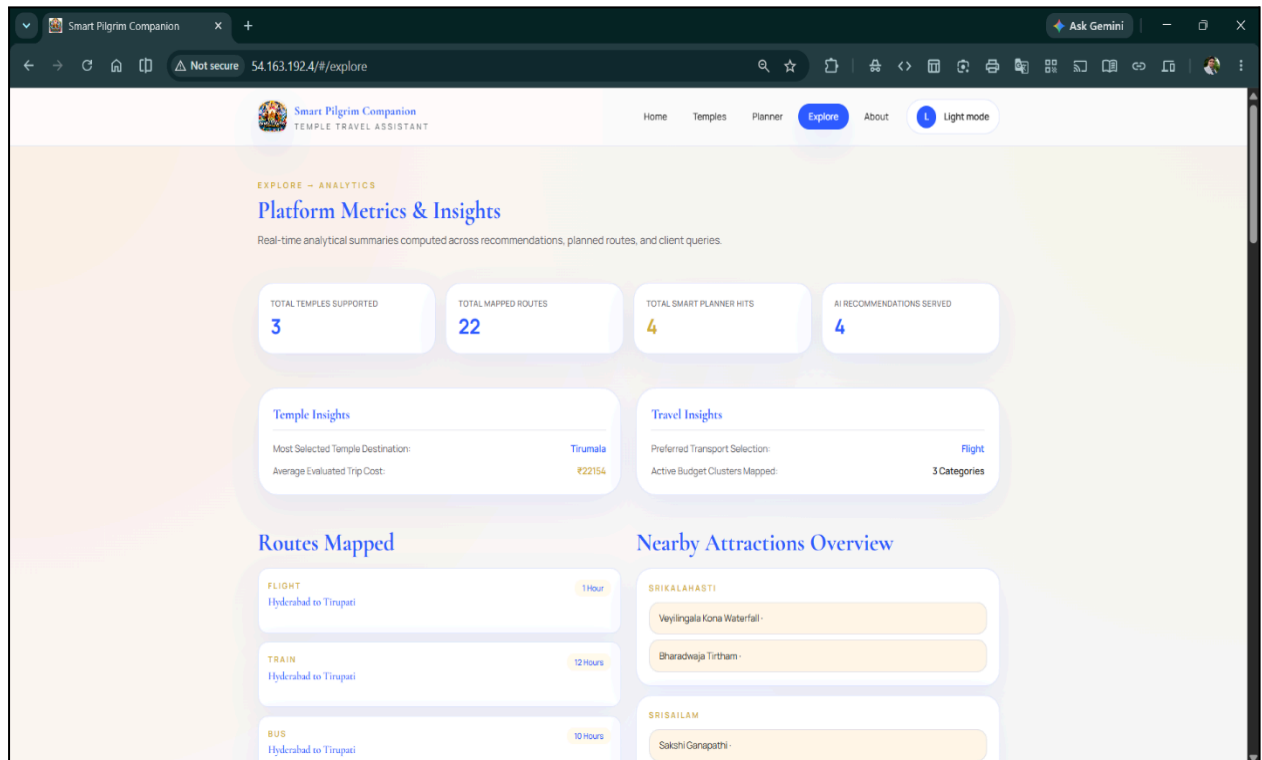


Figure 20: Explore page after AWS deployment

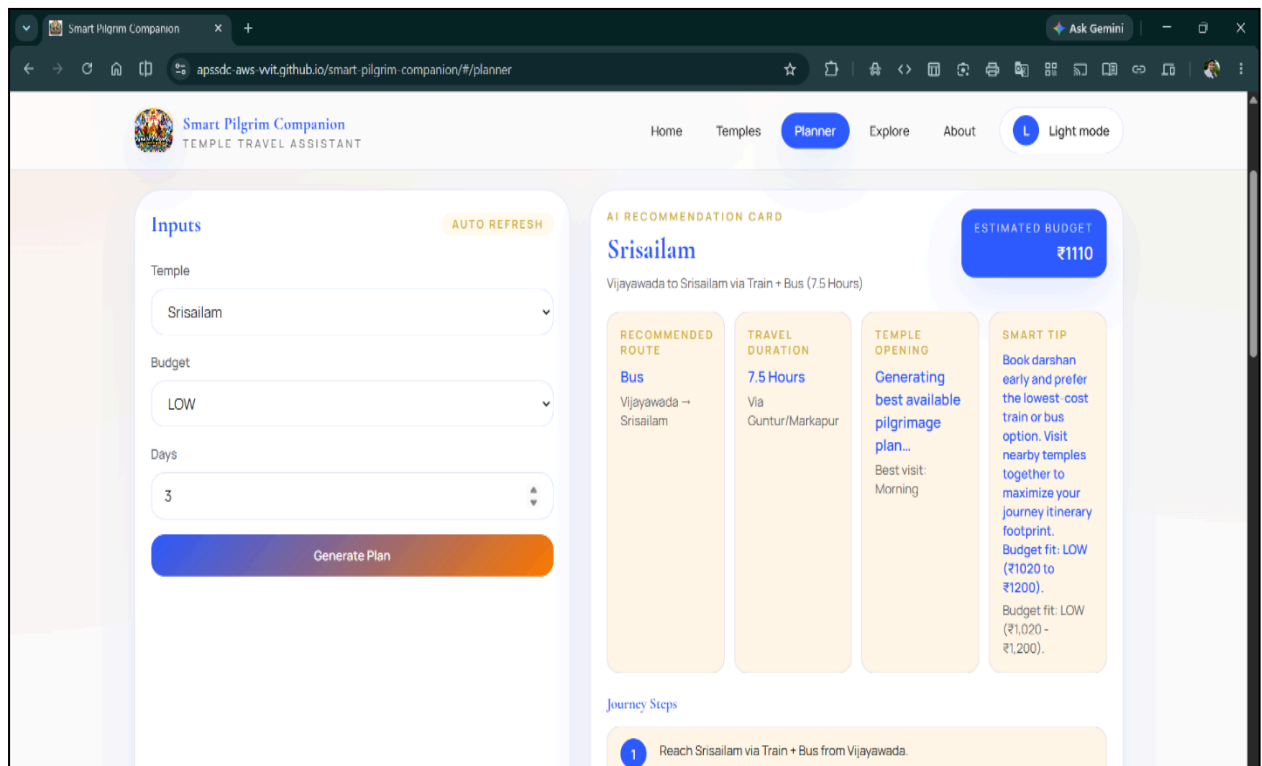


Figure 21: Planner page after AWS deployment

7. AWS Deployment

7.1 Deployment Overview

The project deployment evolved from local development to GitHub frontend hosting and then to AWS infrastructure. The final expected AWS architecture uses EC2, Nginx, Gunicorn, Flask, AWS RDS MySQL, IAM, and CloudWatch.

7.2 Localhost Deployment

During development, the frontend runs through Vite and the backend runs through Flask.

Frontend:

```
cd frontend
npm install
npm run dev
```

Backend:

```
python -m venv venv
venv\Scripts\activate
pip install -r backend\requirements.txt
python database\seed.py
python backend\app.py
```

The frontend service configuration falls back to:

<http://localhost:5000/api>

Localhost evidence:

[INSERT IMAGE: localhost/Sample_UI_UX_Version2_Pics/HomePage.png

Caption: Localhost UI validation home page from sample UI/UX version 2.]

[INSERT IMAGE: localhost/Sample_UI_UX_Version1_Pics/WhatsApp Image 2026-06-23 10.27.16 PM.jpeg Caption: Localhost sample UI/UX version 1 evidence.]

7.3 GitHub Deployment

The frontend supports GitHub Pages deployment using gh-pages.

Important values from frontend/package.json:

- homepage: <https://APSSDC-AWS-VVIT.github.io/smart-pilgrim-companion>
- build: vite build
- predeploy: npm run build
- deploy: gh-pages -d dist

GitHub deployment flow:

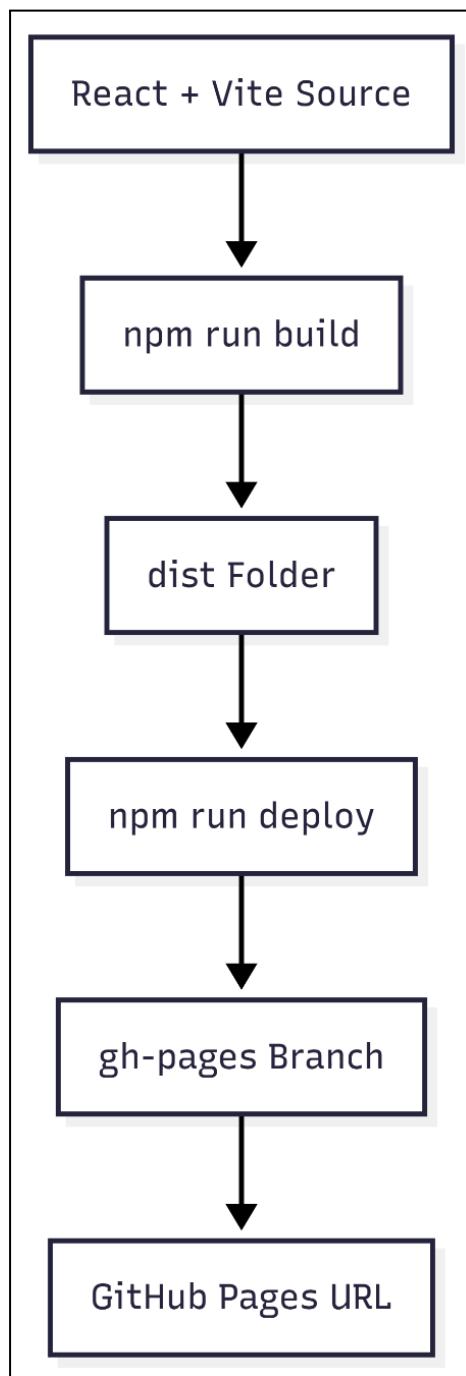


Figure 22: GitHub Deployment Workflow

7.4 AWS Migration Flow



Figure 23: AWS Migration Workflow

7.5 AWS Runtime Architecture

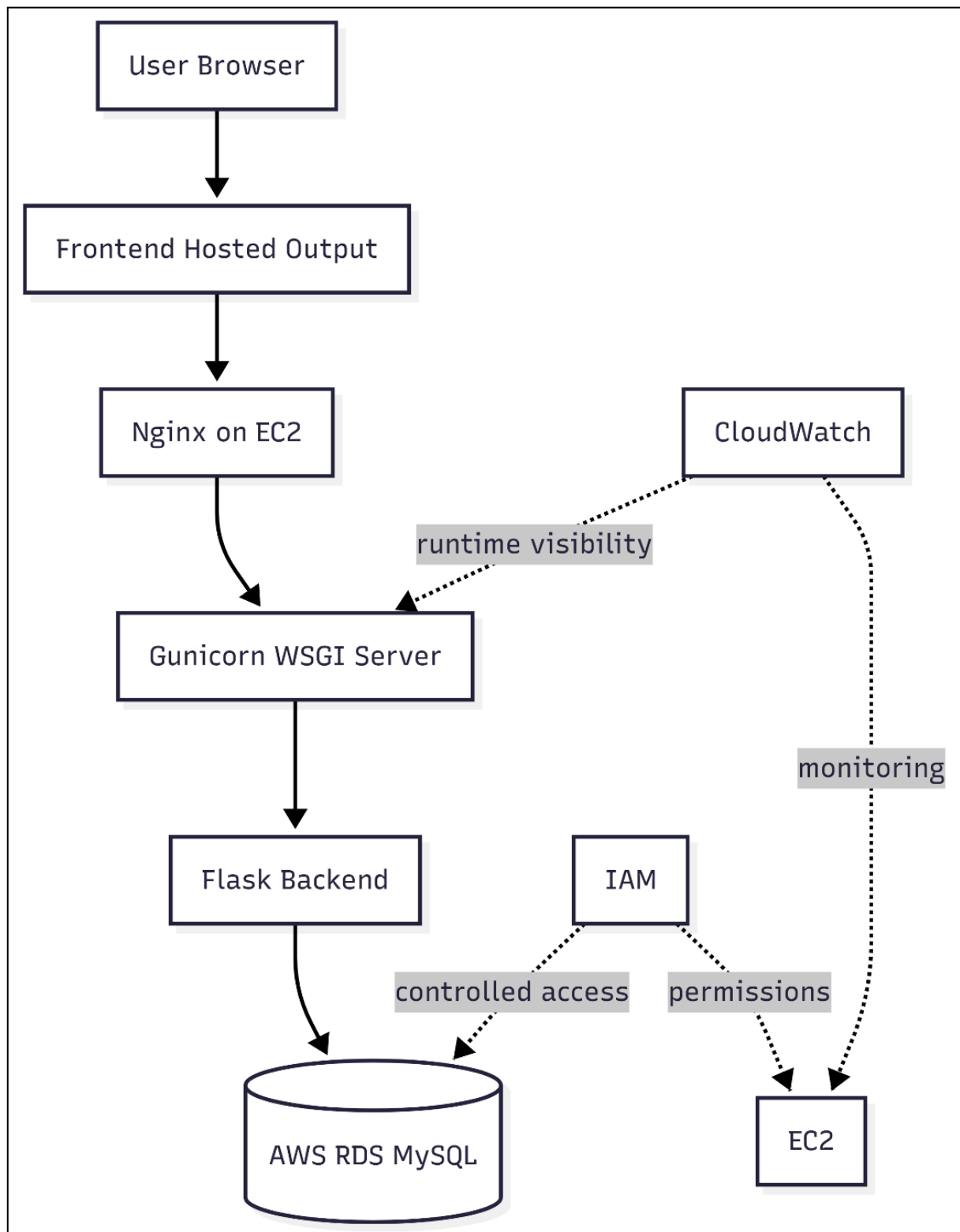


Figure 24: AWS Runtime Architecture

7.6 Backend Environment Configuration

backend/config.py reads the production database connection through:

```
DATABASE_URL
```

If DATABASE_URL is not supplied, the backend falls back to local SQLite:

```
database/smart_pilgrim.db
```

For AWS RDS MySQL deployment, DATABASE_URL should point to the RDS MySQL database. The configuration contains handling for RDS host strings and appends the smart_pilgrim database name when needed.

7.7 Nginx and Gunicorn Role

Nginx receives HTTP traffic and forwards backend requests to Gunicorn. Gunicorn runs the Flask application as a production WSGI service. This separates external request handling from Python application execution

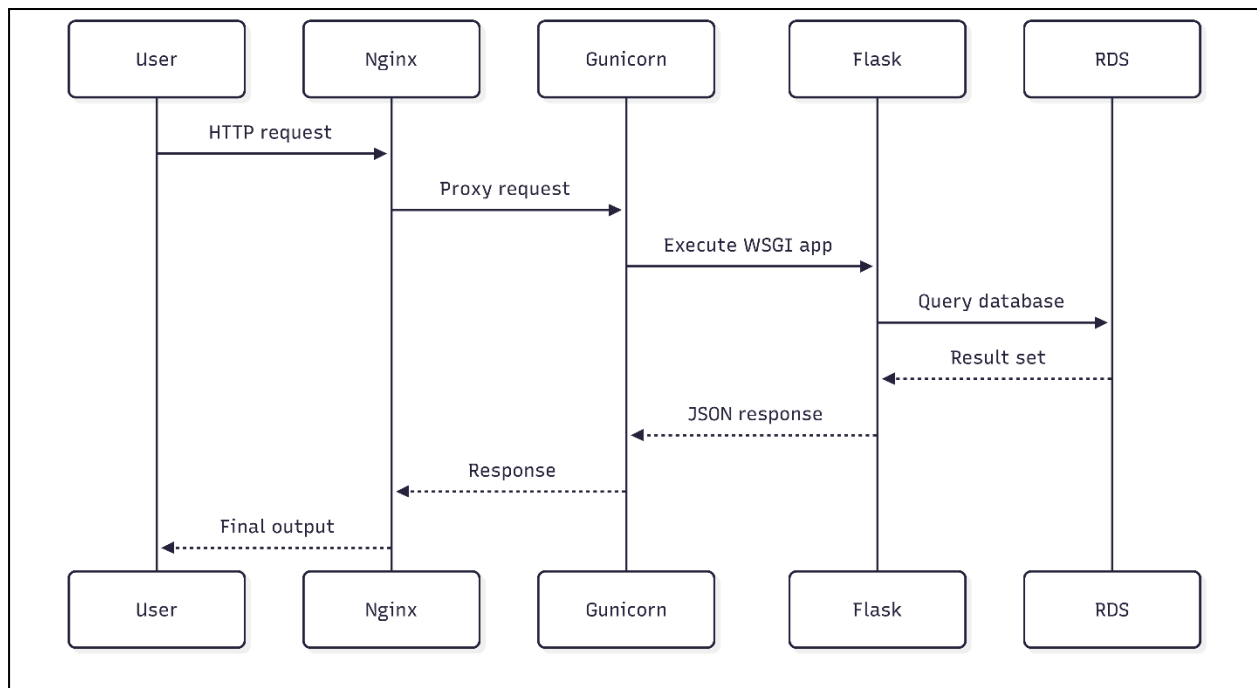


Figure 25: Nginx & Gunicorn Role

7.8 Cloud Monitoring

CloudWatch is used to validate cloud monitoring. Evidence includes EC2 monitoring screenshots and service status screenshots for Unicorn and Nginx.

7.9 AWS Deployment Evidence

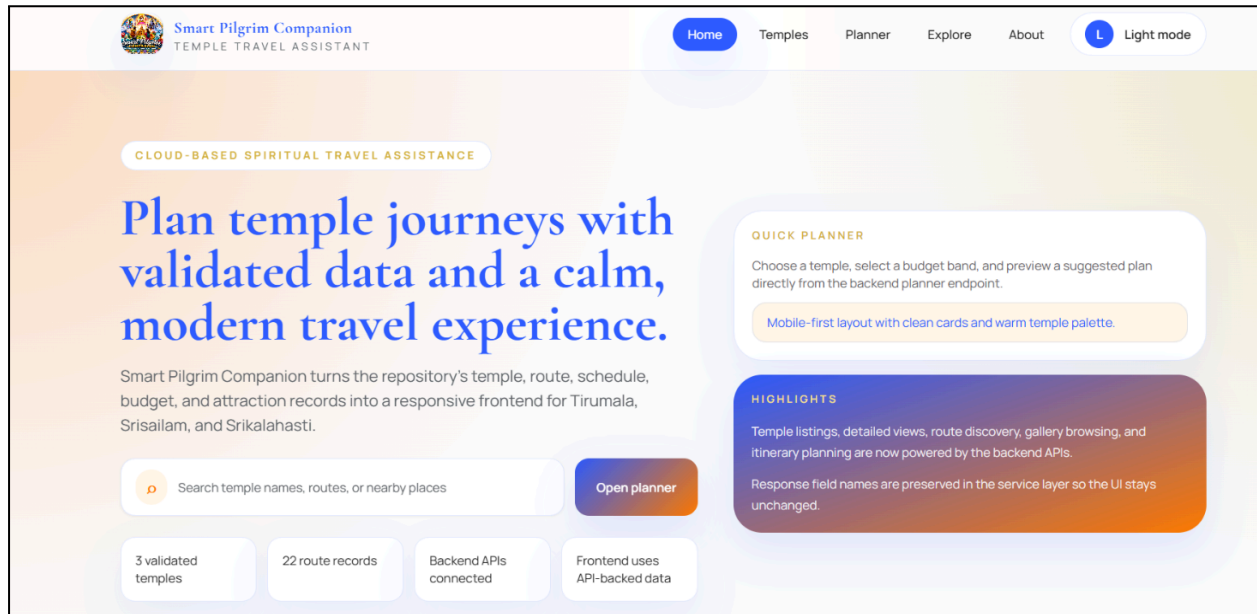


Figure 26: AWS deployment home page

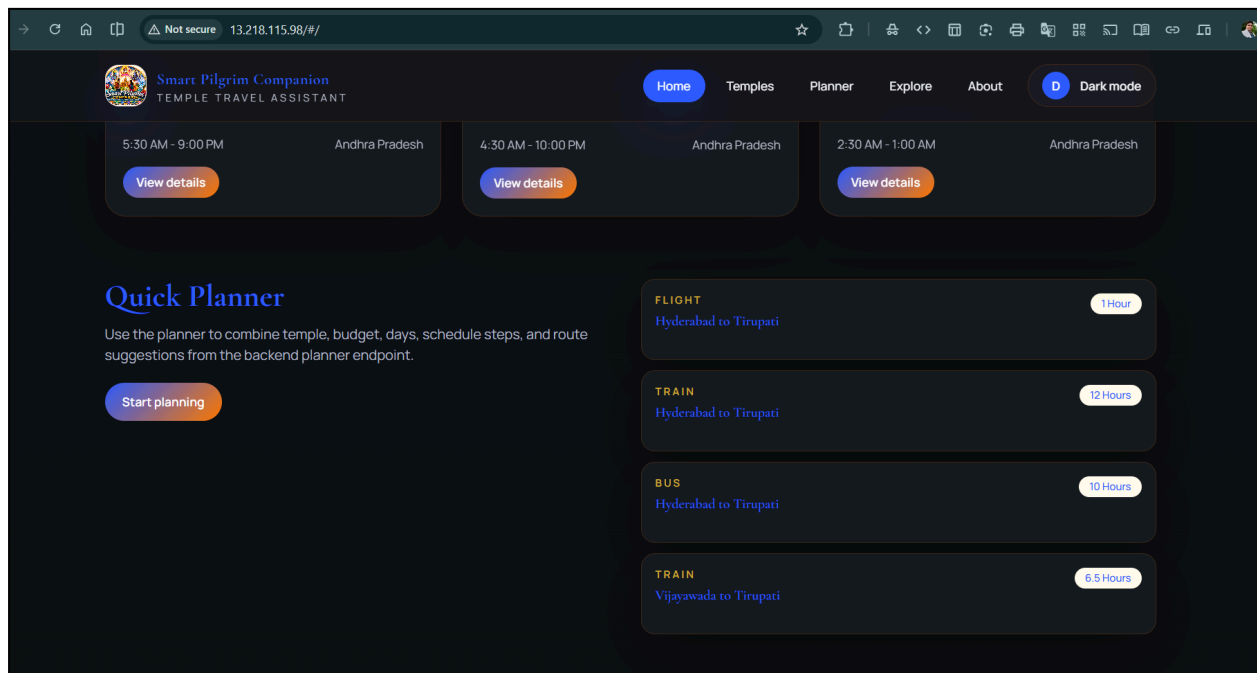


Figure 27: AWS deployment home page in dark mode

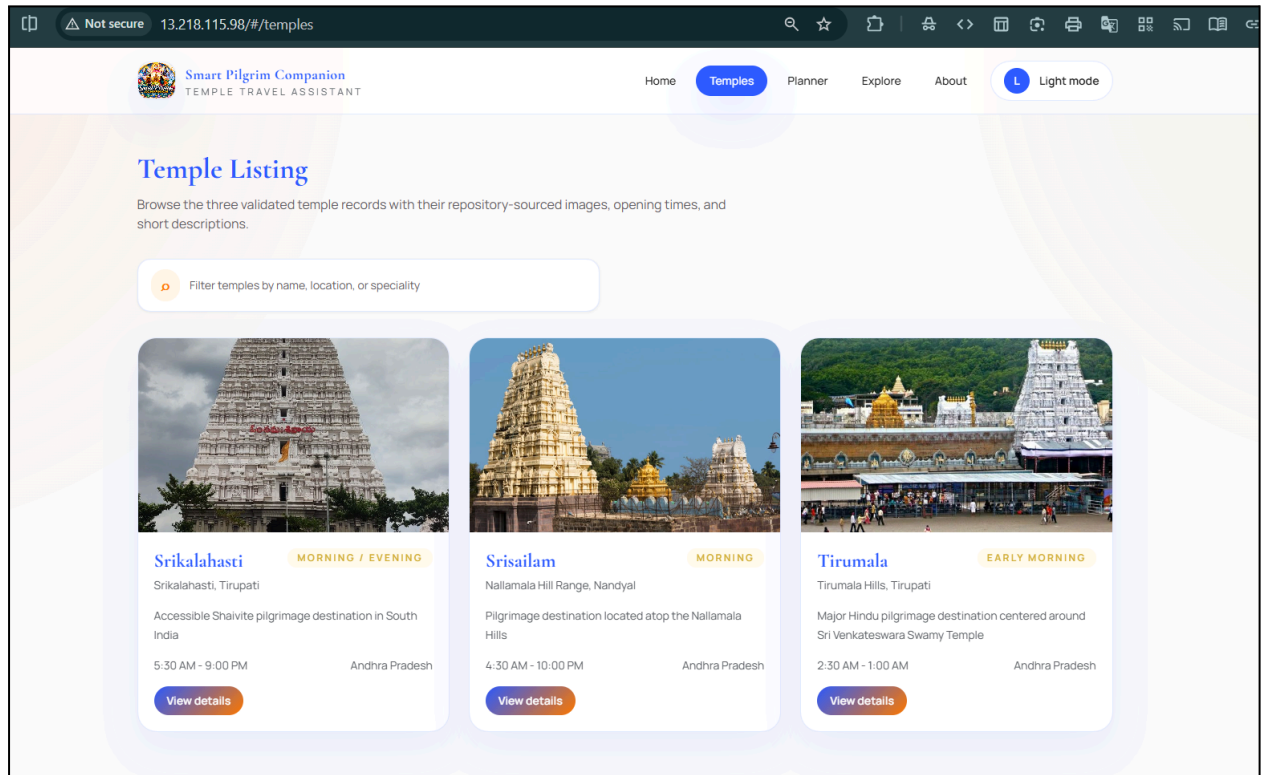


Figure 28: AWS deployment temples page

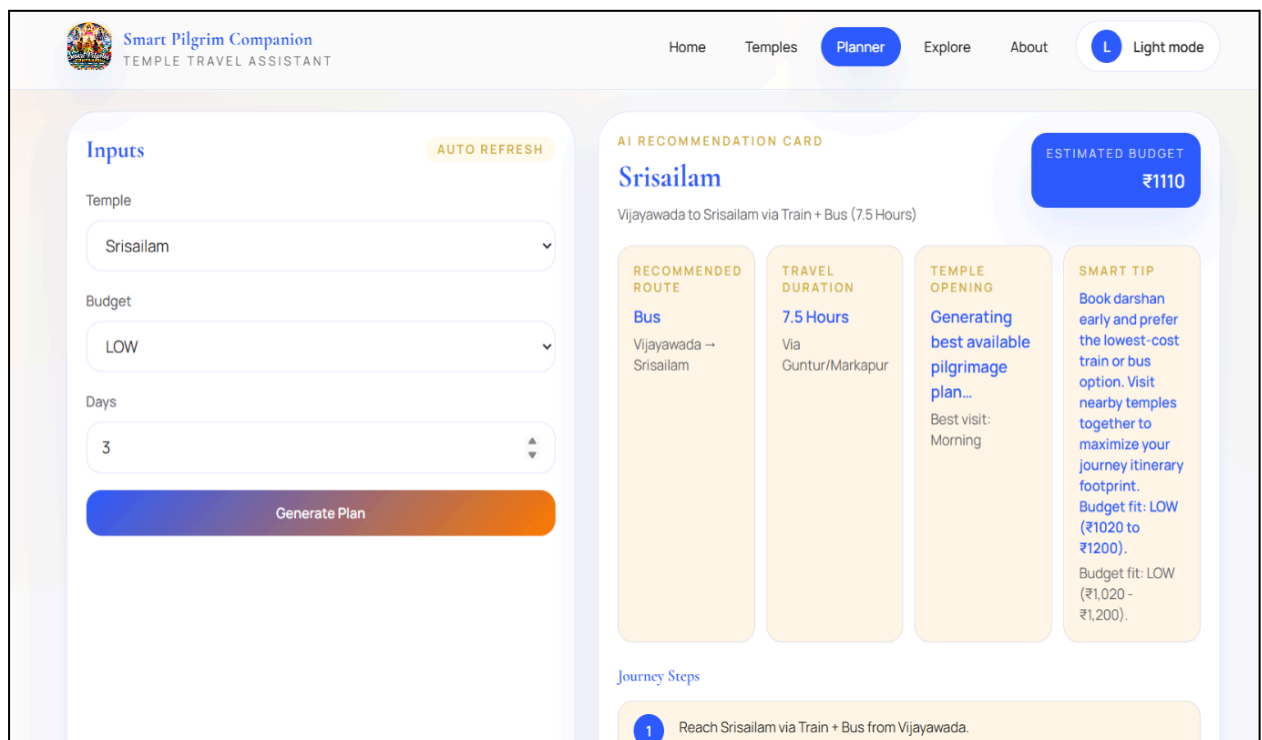


Figure 29: AWS deployment planner page

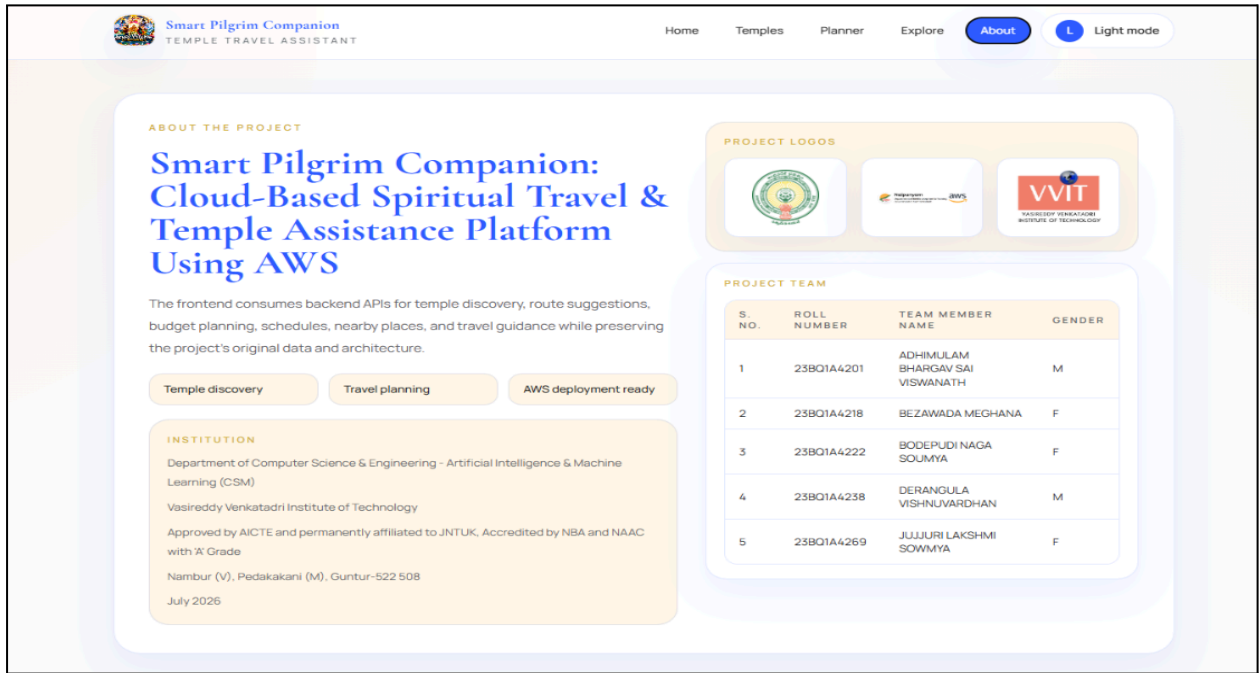


Figure 30: AWS deployment about page

```
aws ubuntu@ip-172-31-29-127:~/smart-pilgrim-companion$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-28 10:06:50 UTC; 1h 6min ago
     Invocation: f3d29ffdbeed4c3ba2f7658687f931ab
       Docs: man:nginx(8)
    Process: 2791 ExecStartPre=/usr/sbin/nginx -t -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 2793 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 2794 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3.2M (peak: 7.5M)
        CPU: 81ms
     CGroup: /system.slice/nginx.service
             └─2794 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
               └─2795 "nginx: worker process"
                 └─2796 "nginx: worker process"

Jun 28 10:06:50 ip-172-31-29-127 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jun 28 10:06:50 ip-172-31-29-127 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-29-127:~/smart-pilgrim-companion$
```

Figure 31: Nginx service status on Amazon

```
aws ubuntu@ip-172-31-29-127:~/smart-pilgrim-companion$ ps aux | grep gunicorn
ubuntu 2093 0.0 2.6 115772 24496 ? Ssl 09:18 0:01 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/python3 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/guni
corn --workers 3 --bind 127.0.0.1:8000 app:app
ubuntu 2098 0.0 7.8 98904 72952 ? S 09:18 0:02 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/python3 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/guni
corn --workers 3 --bind 127.0.0.1:8000 app:app
ubuntu 2099 0.0 7.7 98880 72444 ? S 09:18 0:02 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/python3 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/guni
corn --workers 3 --bind 127.0.0.1:8000 app:app
ubuntu 2100 0.0 7.7 98972 72396 ? S 09:18 0:02 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/python3 /home/ubuntu/smart-pilgrim-companion/backend/venv/bin/guni
corn --workers 3 --bind 127.0.0.1:8000 app:app
ubuntu 3190 0.0 0.2 7276 2316 pts/0 S+ 11:14 0:00 grep --color=auto gunicorn
ubuntu@ip-172-31-29-127:~/smart-pilgrim-companion$
```

Figure 32: Gunicorn service status on Amazon EC2

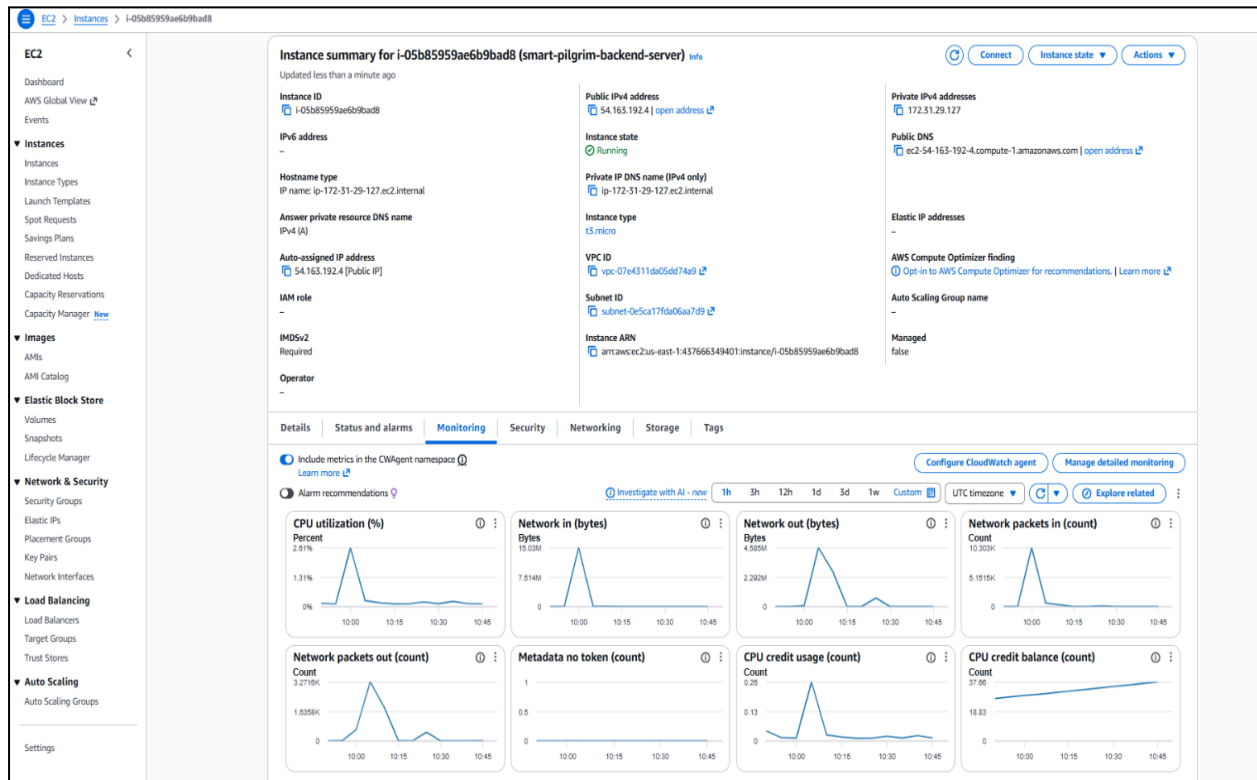


Figure 33: Amazon EC2 monitoring dashboard

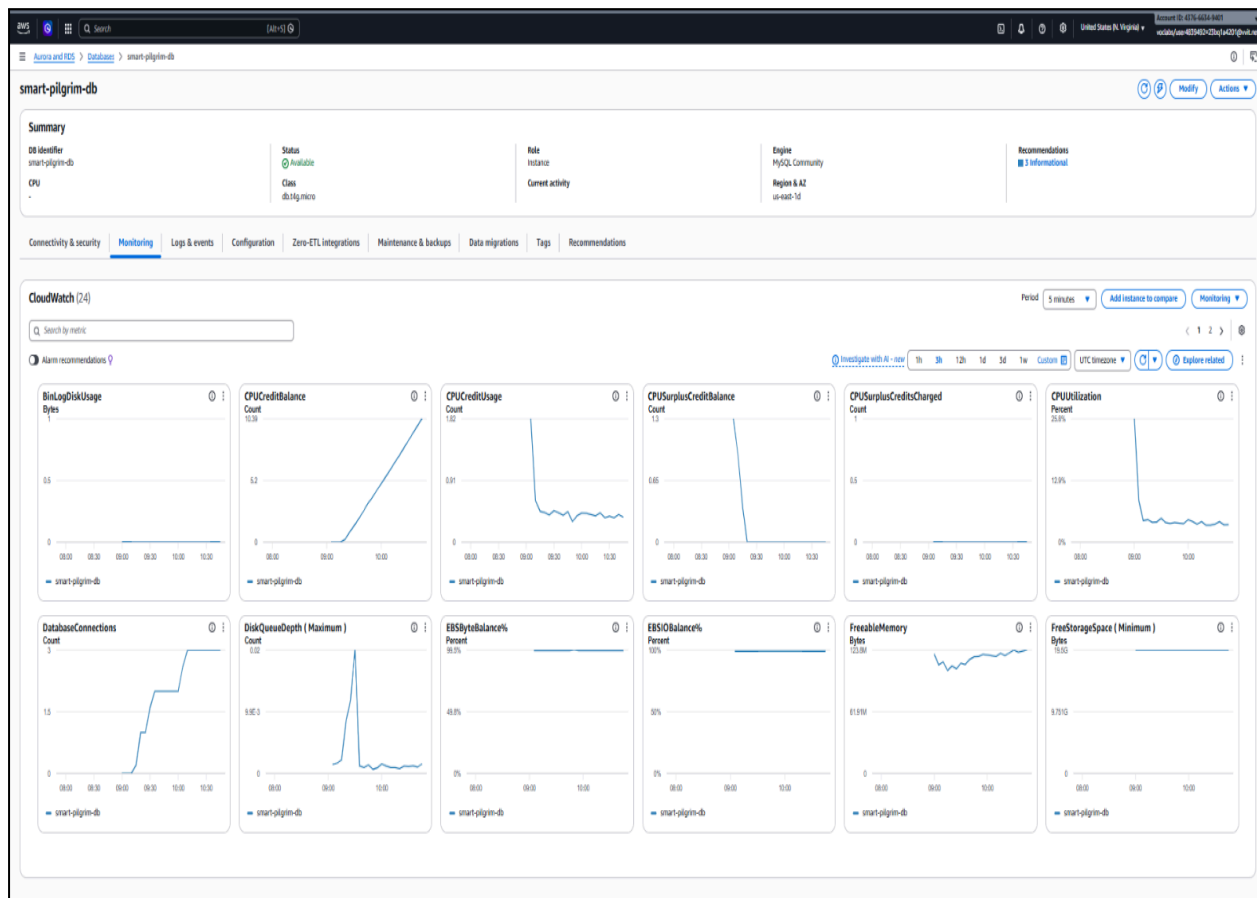


Figure 34: Amazon RDS MySQL configuration

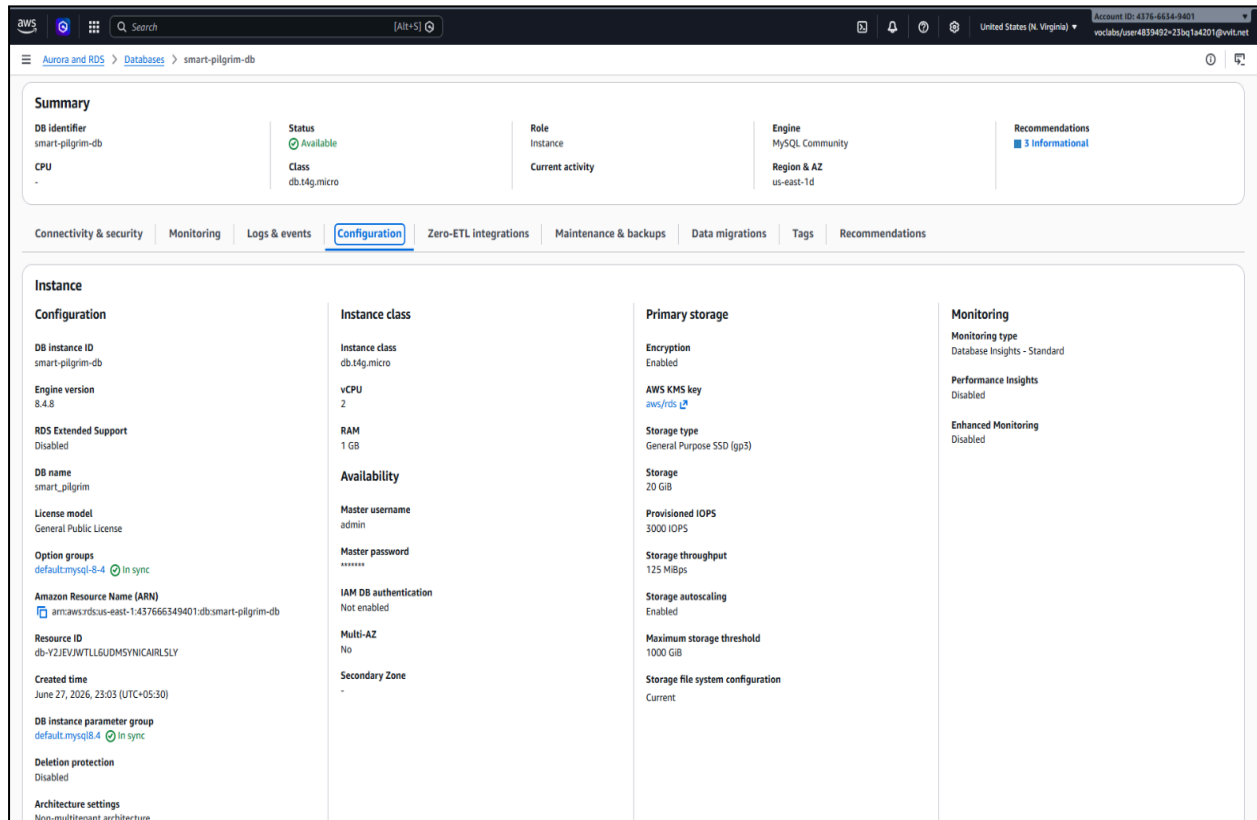


Figure 35: Amazon RDS monitoring dashboard

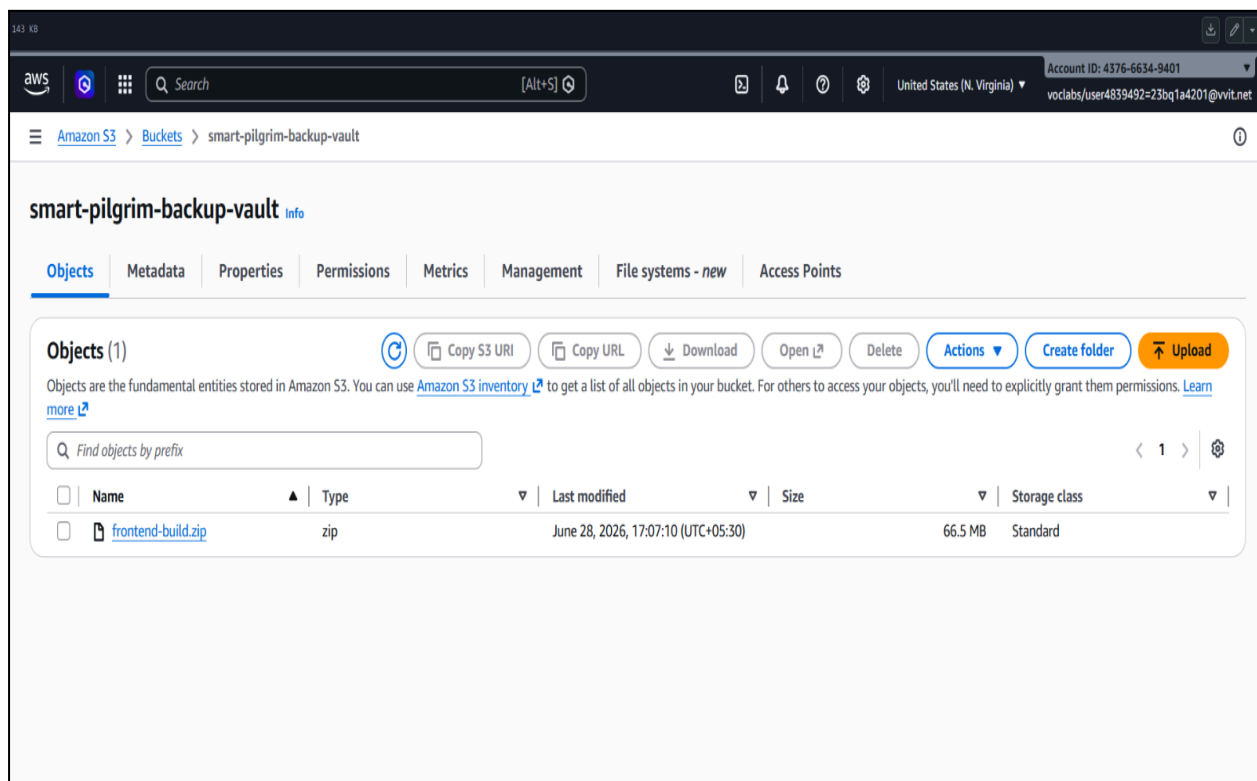


Figure 36: Amazon RDS security group configuration

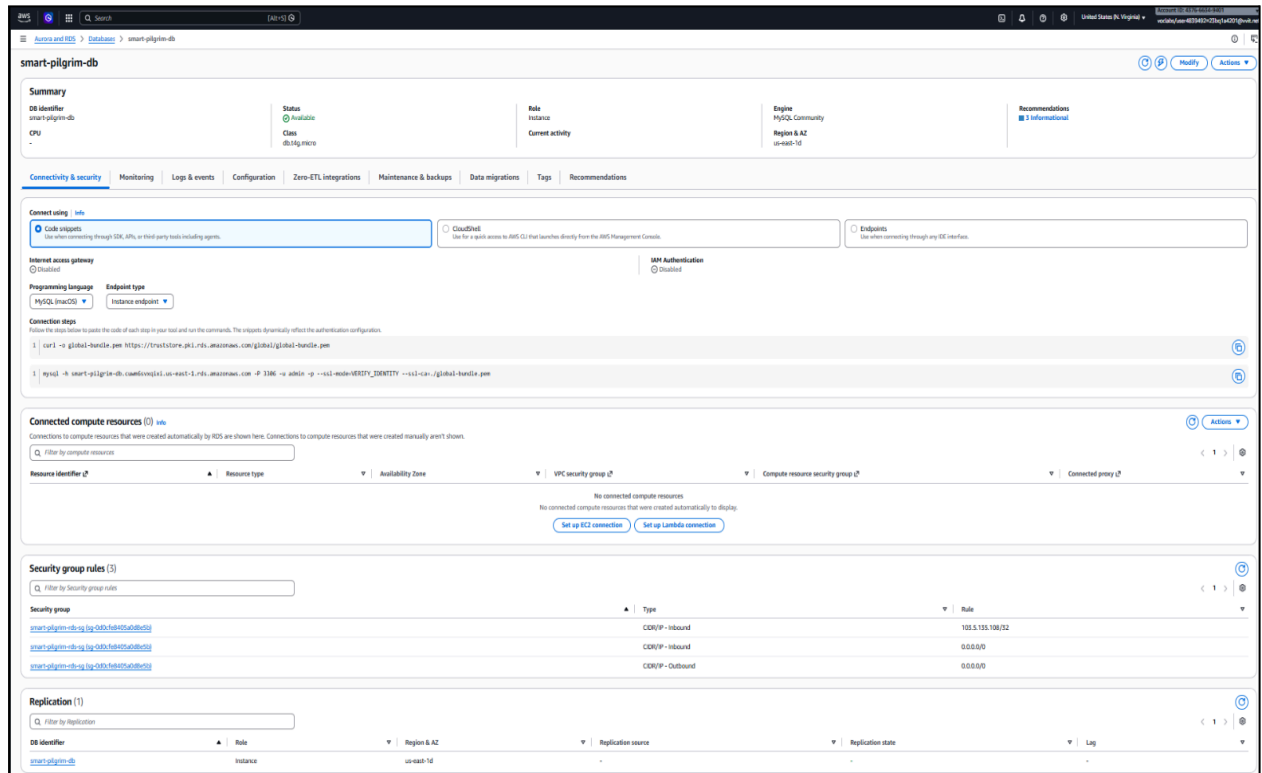


Figure 37: Amazon EC2 security group configuration

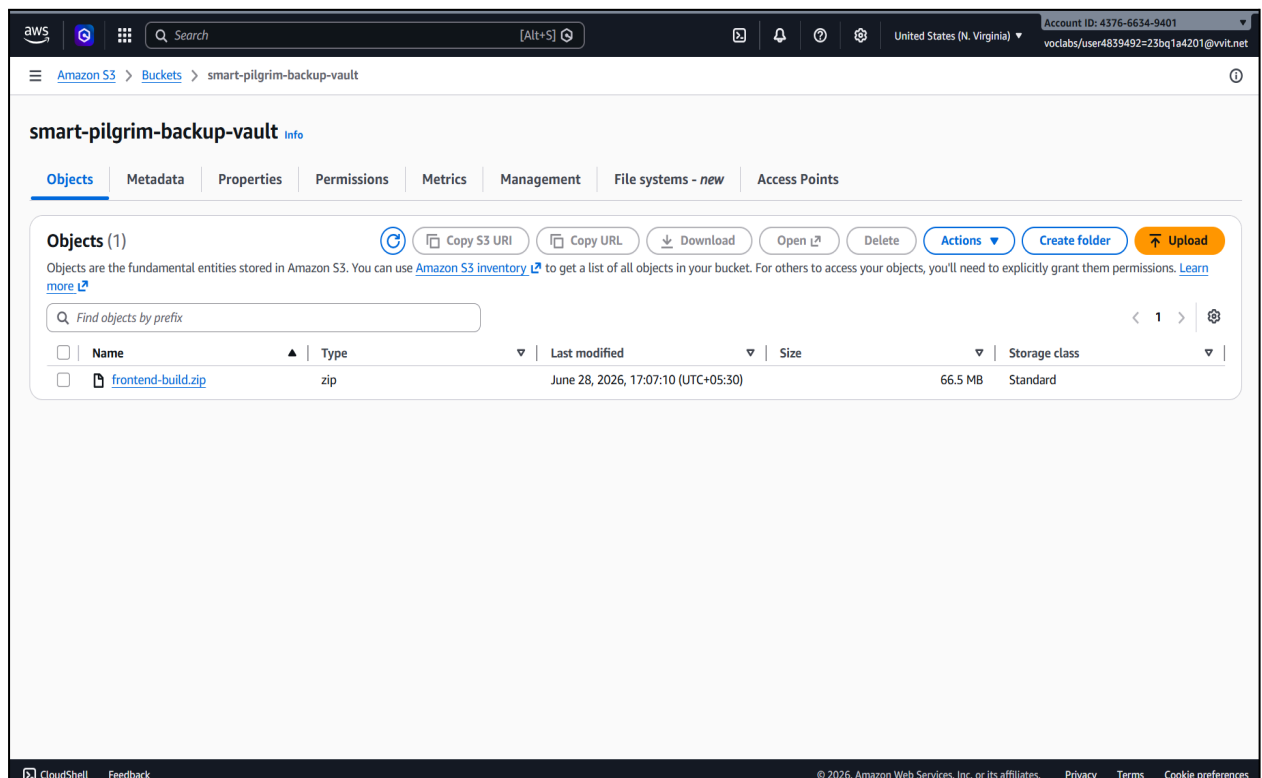


Figure 38: Amazon S3 frontend ZIP file upload

8: Testing and Results

8.1 Testing Overview

Testing focused on frontend functionality, backend API behavior, deployment validation, cloud resource validation, and final output verification.

8.2 Functional Testing

Functional testing validates the main user workflows:

- Open the home page.
- Navigate to temples.
- Open temple details.
- Explore destination information.
- Use the planner page.
- Trigger planner and recommendation API calls.
- View route, budget, timeline, nearby places, tips, and risk notes.

8.3 API Testing

Backend endpoints verified from the repository include:

- `GET /api/health`
- `GET /api/temples`
- `GET /api/temples/<identifier>`
- `GET /api/routes`
- `GET /api/planner`
- `GET /api/recommendation`
- `GET /api/performance`
- `GET /api/analytics/summary`

Validation behavior includes:

- Missing temple selection returns an error.
- Invalid days values return an error.
- Invalid budget selection returns an error.
- Unknown temple returns a not found response.

8.4 Deployment Validation

Deployment validation covered:

- Frontend build generation through `npm run build`.

- GitHub Pages deployment through `npm run deploy`.
- AWS frontend page access.
- EC2 service accessibility.
- Nginx status validation.
- Gunicorn status validation.
- RDS configuration and monitoring evidence.
- CloudWatch/EC2 monitoring evidence.

8.5 Cloud Validation

Cloud validation confirms the final AWS migration path:

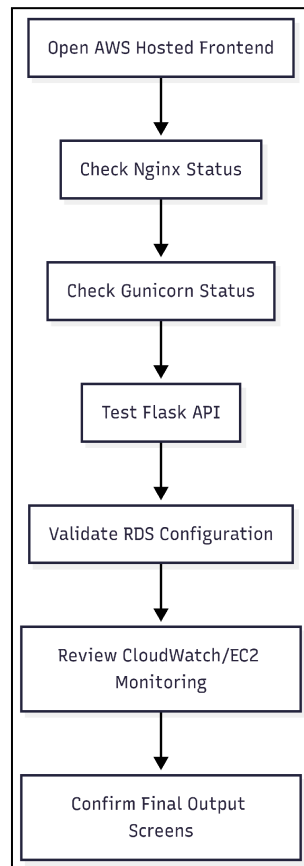


Figure 39: Cloud Validation

8.6 Testing Flow

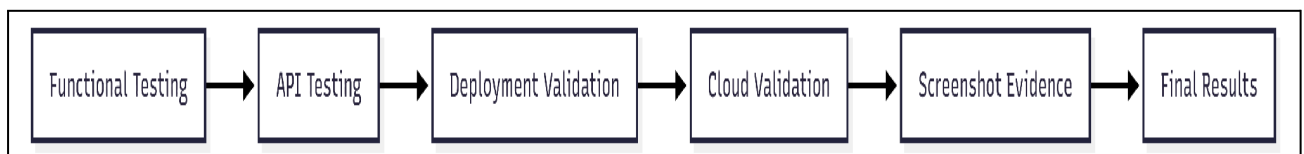


Figure 40: Testing Flow

8.7 Results

The project produced a working frontend, backend API layer, planner workflow, recommendation response, deployment evidence, monitoring evidence, and final output screenshots. The implementation demonstrates the expected academic project objectives:

- Full-stack implementation completed.
- Localhost development workflow validated.
- GitHub Pages frontend deployment evidenced.
- AWS EC2 deployment evidenced.
- Nginx and Gunicorn service status evidenced.
- RDS configuration and monitoring evidenced.
- Team contribution and final output evidence collected.

8.8 Testing Evidence Screenshots

The screenshot displays the 'Smart Pilgrim Companion' web application, which serves as a 'TEMPLE TRAVEL ASSISTANT'. The interface includes a navigation bar with links for Home, Temples, Planner (highlighted), Explore, and About, along with a Light mode toggle. The main heading is 'Intelligent pilgrimage assistant for temple journeys', with a subtext describing it as a 'Centralized intelligent coordinate tracking engine mapping backend routes, metrics and validation criteria dynamically.' The 'Inputs' section on the left allows users to select a temple (Tirumala), set a budget (LOW), and specify the number of days (3). An 'AUTO REFRESH' button and a 'Generate Plan' button are also present. The 'AI RECOMMENDATION CARD' on the right provides details for the 'Tirumala' route from Vijayawada to Tirupati via Train (6.5 Hours). It includes a recommended route (Train), travel duration (6.5 Hours), temple opening information, and a smart tip about budgeting. The estimated budget is shown as ₹3950.

RECOMMENDED ROUTE	TRAVEL DURATION	TEMPLE OPENING	SMART TIP
Train Vijayawada → Tirupati	6.5 Hours Pinakini Express	Generating best available pilgrimage plan... Best visit: Early Morning	Prefer train under low budget networks and arrange early morning darshan transitions. Visit nearby temples together to maximize your journey/itinerary footprint. Budget fit: LOW (₹3400 to ₹4500). Budget fit: LOW

Figure 41: Testing evidence output 1

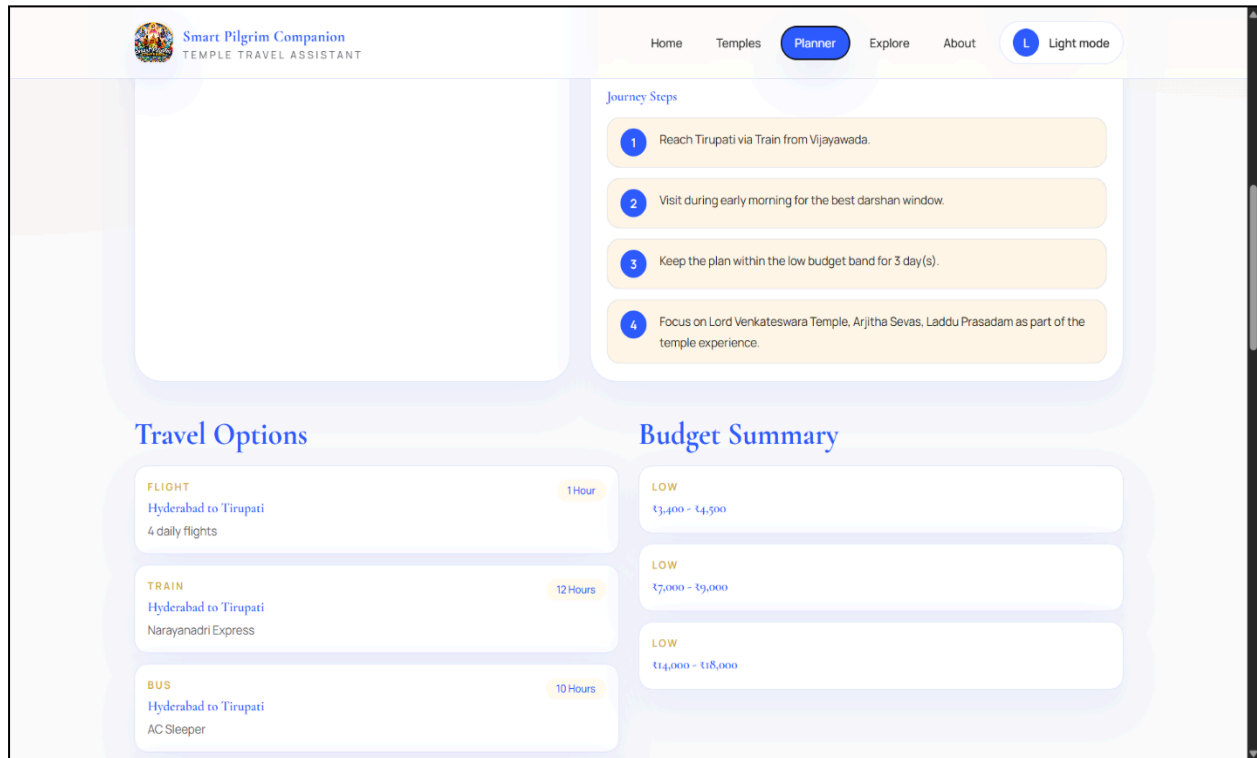


Figure 42: Testing evidence output 2

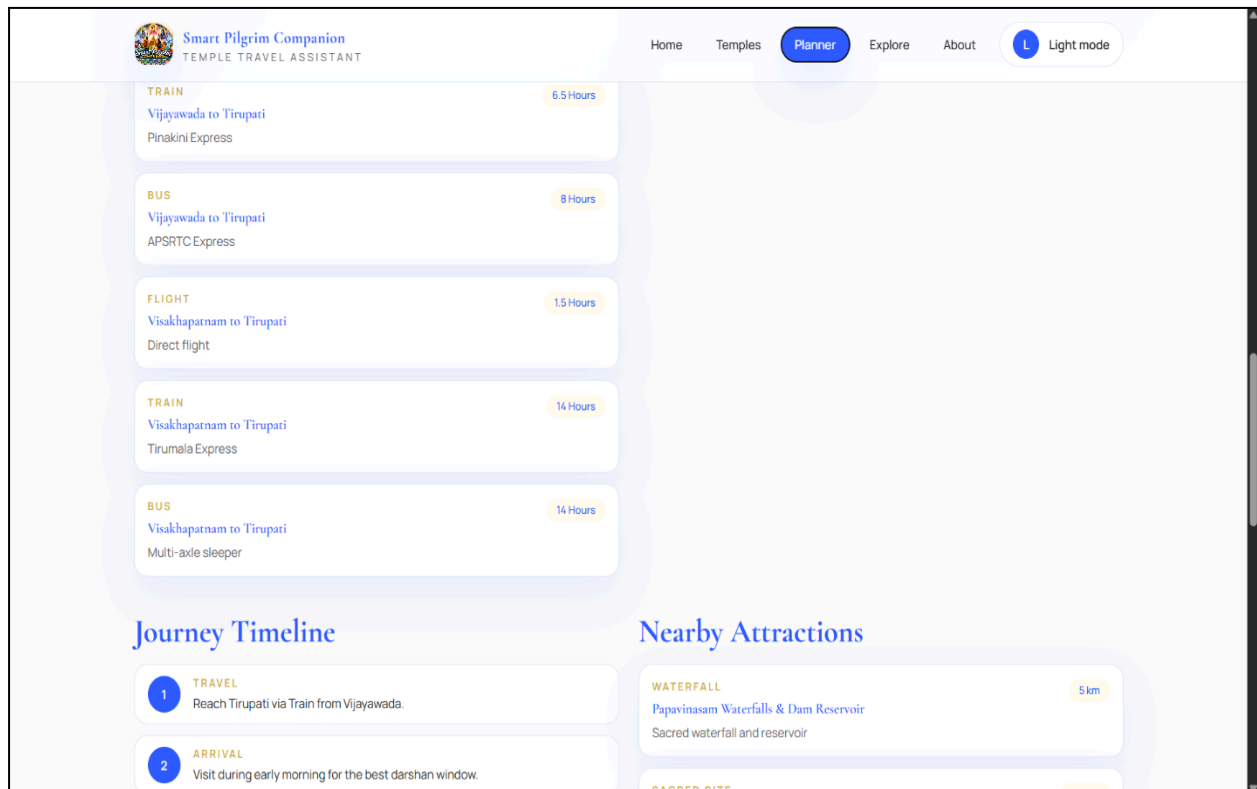


Figure 43: Testing evidence output 3

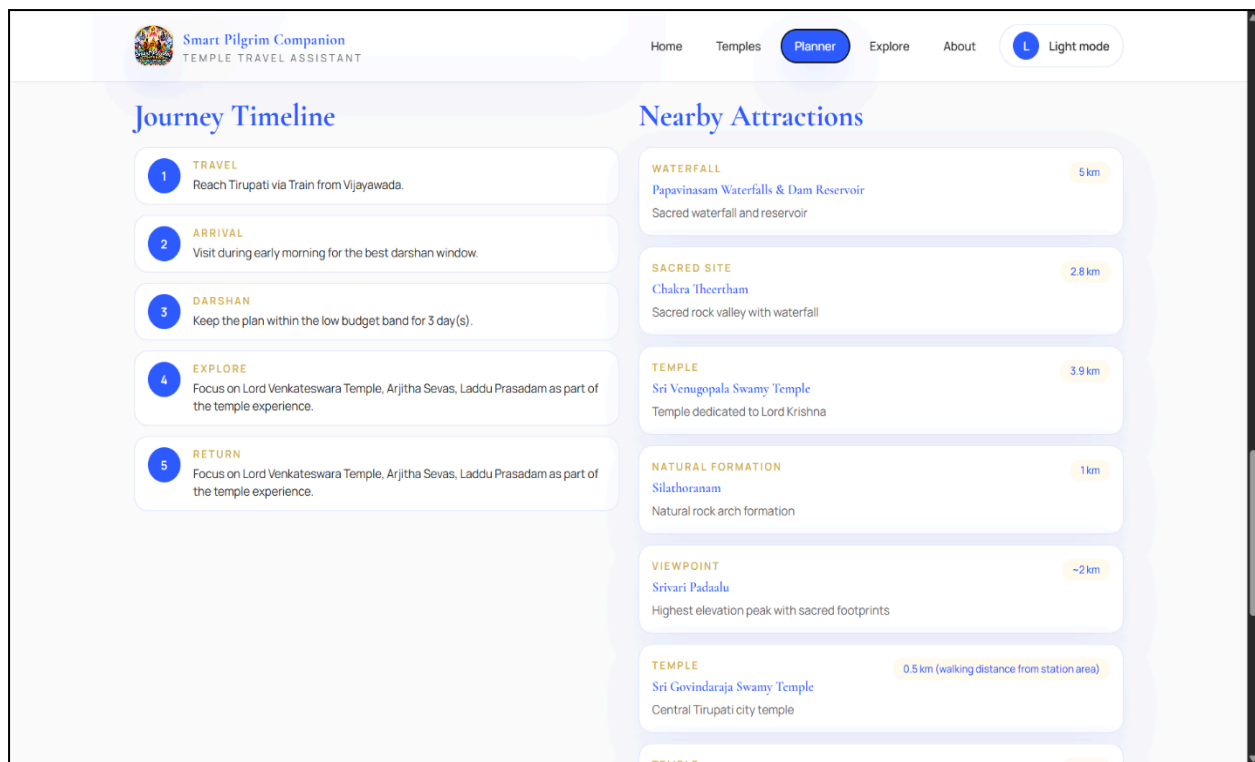


Figure 44: Testing evidence output 4

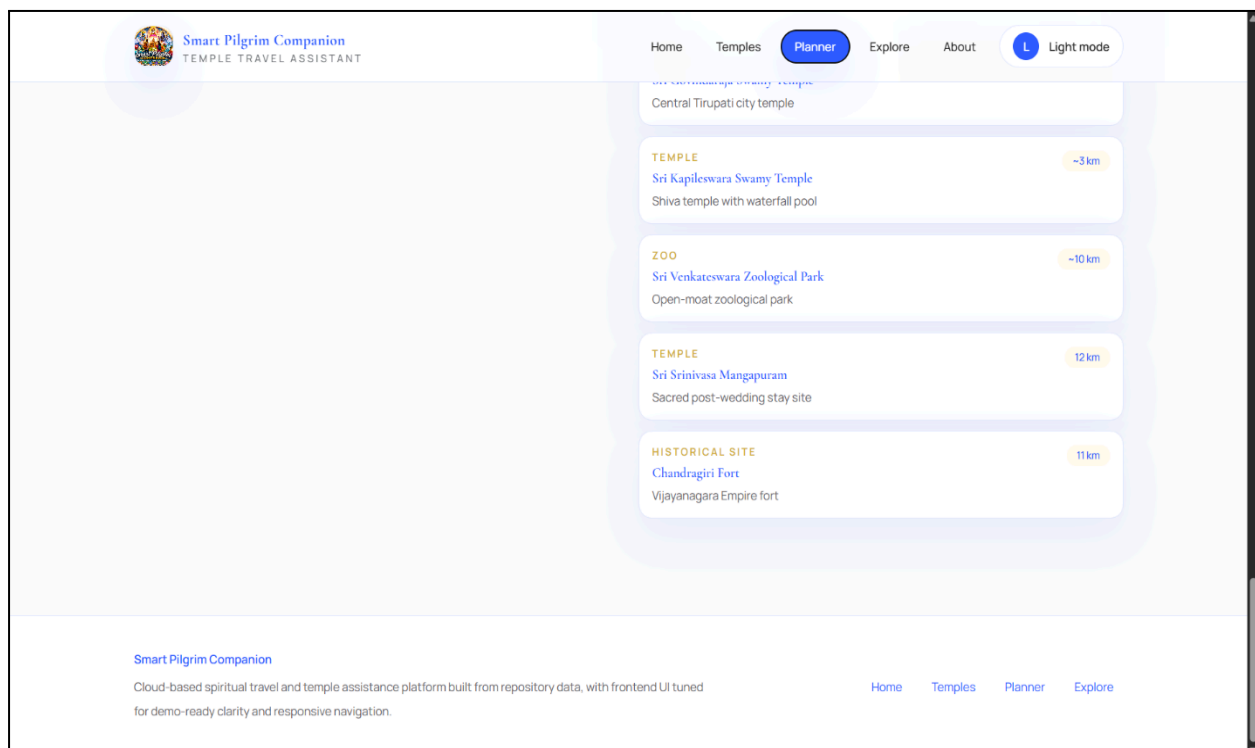


Figure 45: Testing evidence output 5

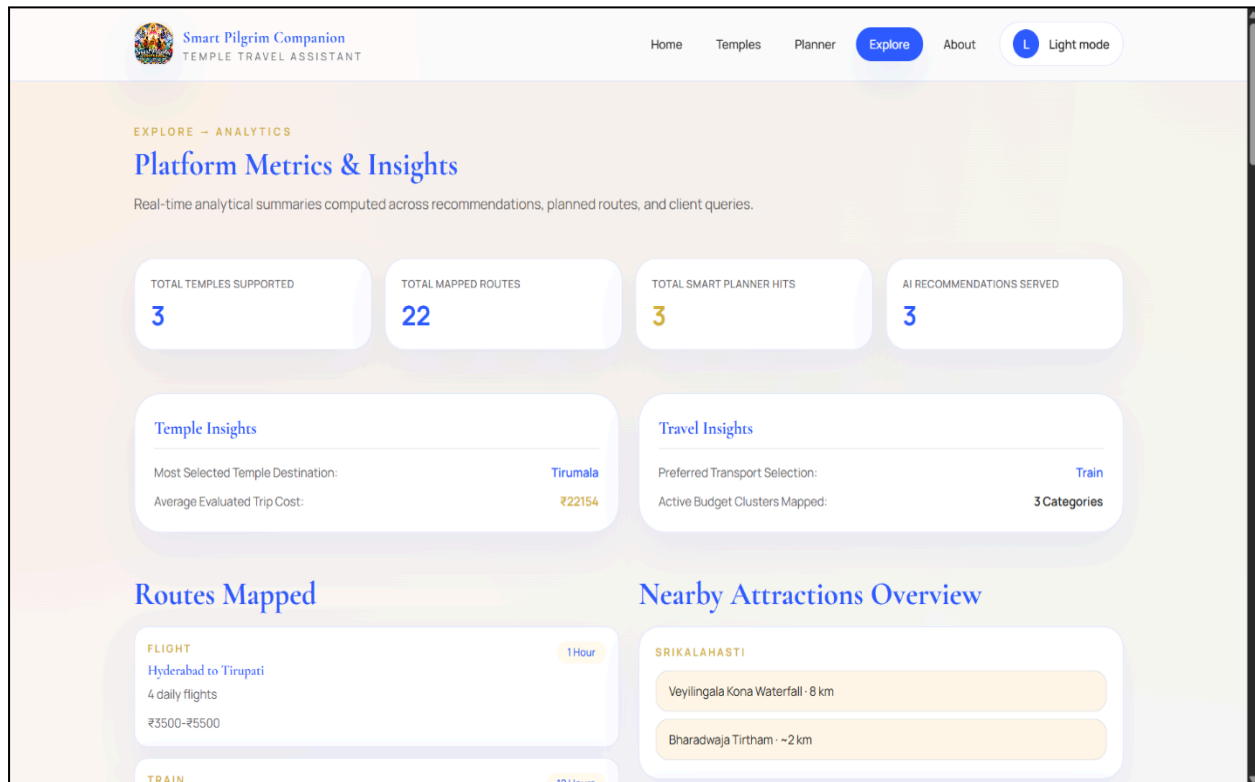


Figure 46: Testing evidence output 6

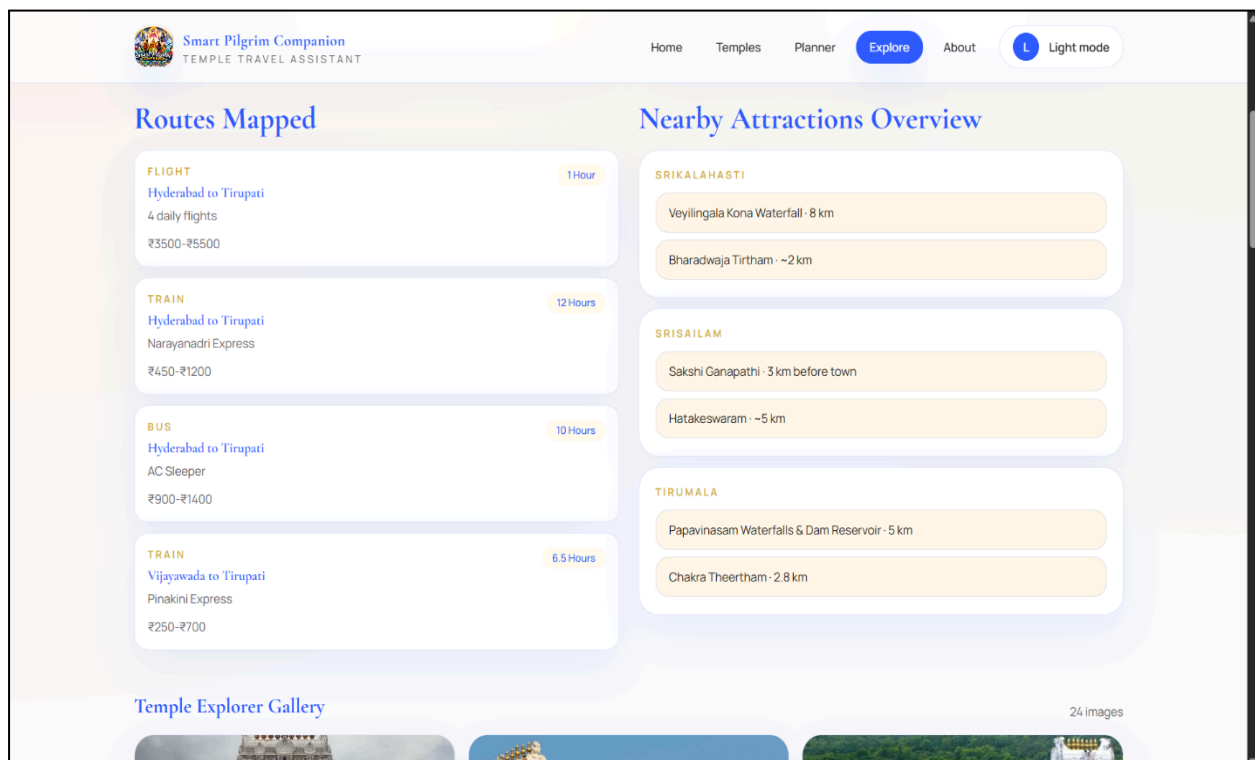


Figure 47: Testing evidence output 7

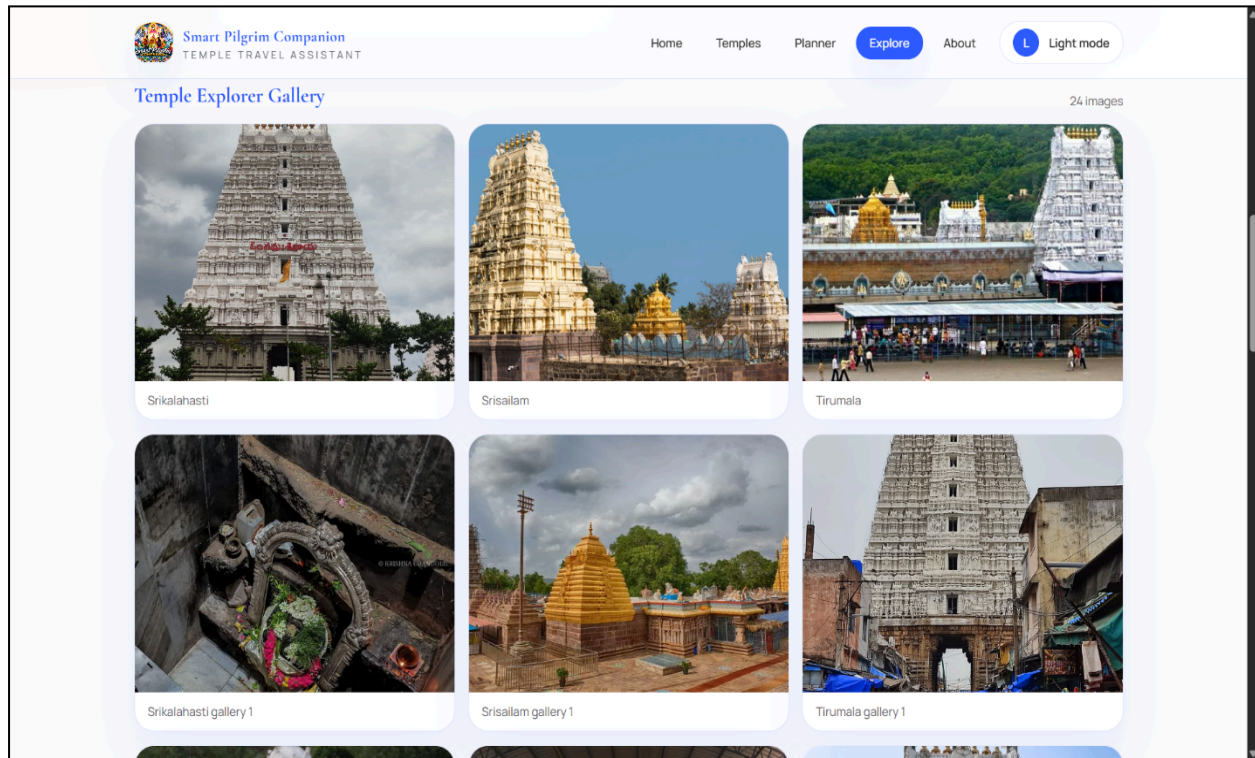


Figure 48: Testing evidence output 8

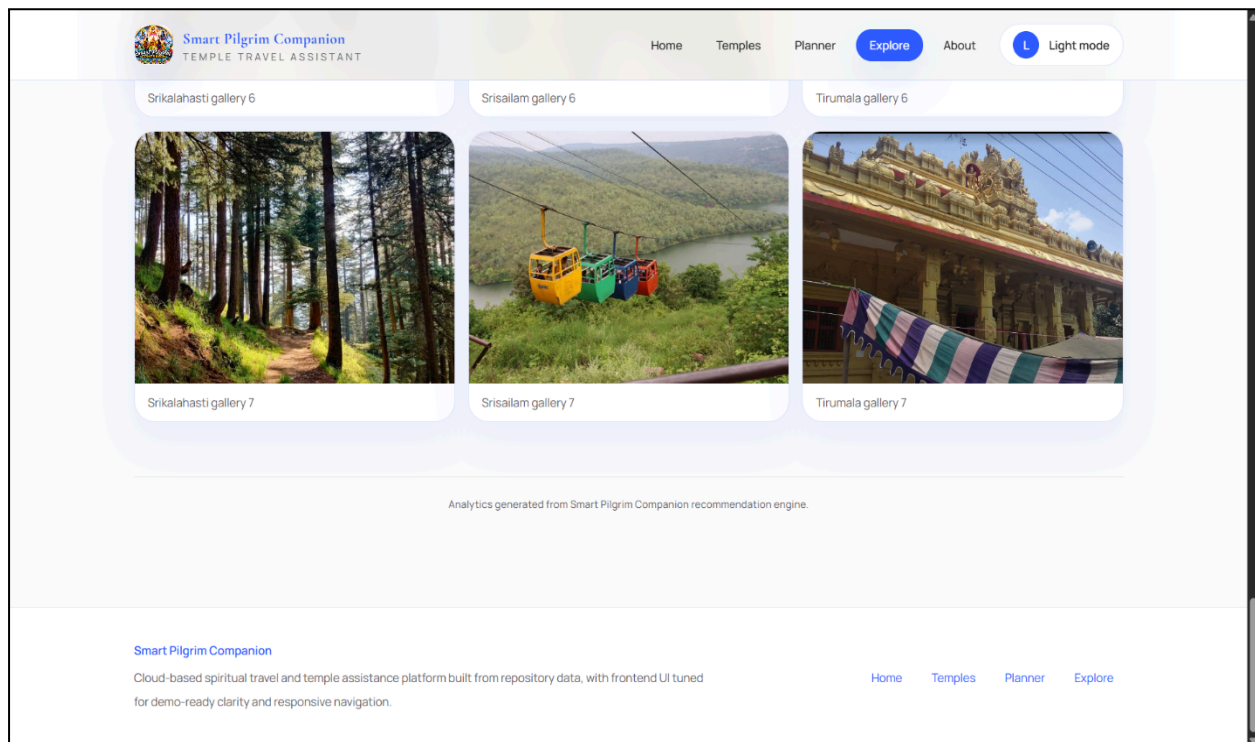


Figure 49: Testing evidence output 10

8.9 Final Output Screenshots

The screenshot displays the 'Smart Pilgrim Companion' web application interface. The header includes the logo, navigation links (Home, Temples, Planner, Explore, About), and a 'Light mode' toggle. The main heading is 'Intelligent pilgrimage assistant for temple journeys', with a subtext: 'Centralized intelligent coordinate tracking engine mapping backend routes, metrics and validation criteria dynamically.'

The 'Inputs' section on the left contains the following fields:

- Temple: Tirumala (dropdown menu)
- Budget: LOW (dropdown menu)
- Days: 3 (input field)
- Generate Plan button
- AUTO REFRESH button

The 'AI RECOMMENDATION CARD' on the right is titled 'Tirumala' and shows the route 'Vijayawada to Tirupati via Train (6.5 Hours)'. It includes an 'ESTIMATED BUDGET' of ₹3950. The card is divided into four columns:

- RECOMMENDED ROUTE:** Train, Vijayawada → Tirupati
- TRAVEL DURATION:** 6.5 Hours, Pinakini Express
- TEMPLE OPENING:** Generating best available pilgrimage plan... Best visit: Early Morning
- SMART TIP:** Prefer train under low budget networks and arrange early morning darshan transitions. Visit nearby temples together to maximize your journey itinerary footprint. Budget fit: LOW (₹3400 to ₹4500). Budget fit: LOW

Figure 50: Final output screen 1

The screenshot displays the 'Smart Pilgrim Companion' web application interface, showing the 'Journey Steps' and 'Budget Summary' sections.

The 'Journey Steps' section lists four steps:

- 1 Reach Tirupati via Train from Vijayawada.
- 2 Visit during early morning for the best darshan window.
- 3 Keep the plan within the low budget band for 3 day(s).
- 4 Focus on Lord Venkateswara Temple, Arjitha Sevas, Laddu Prasadam as part of the temple experience.

The 'Travel Options' section lists three options:

- FLIGHT:** Hyderabad to Tirupati, 4 daily flights, 1 Hour
- TRAIN:** Hyderabad to Tirupati, Narayanadri Express, 12 Hours
- BUS:** Hyderabad to Tirupati, AC Sleeper, 10 Hours

The 'Budget Summary' section shows three budget bands:

- LOW:** ₹3,400 - ₹4,500
- LOW:** ₹7,000 - ₹9,000
- LOW:** ₹4,000 - ₹8,000

Figure 51: Final output screen 2

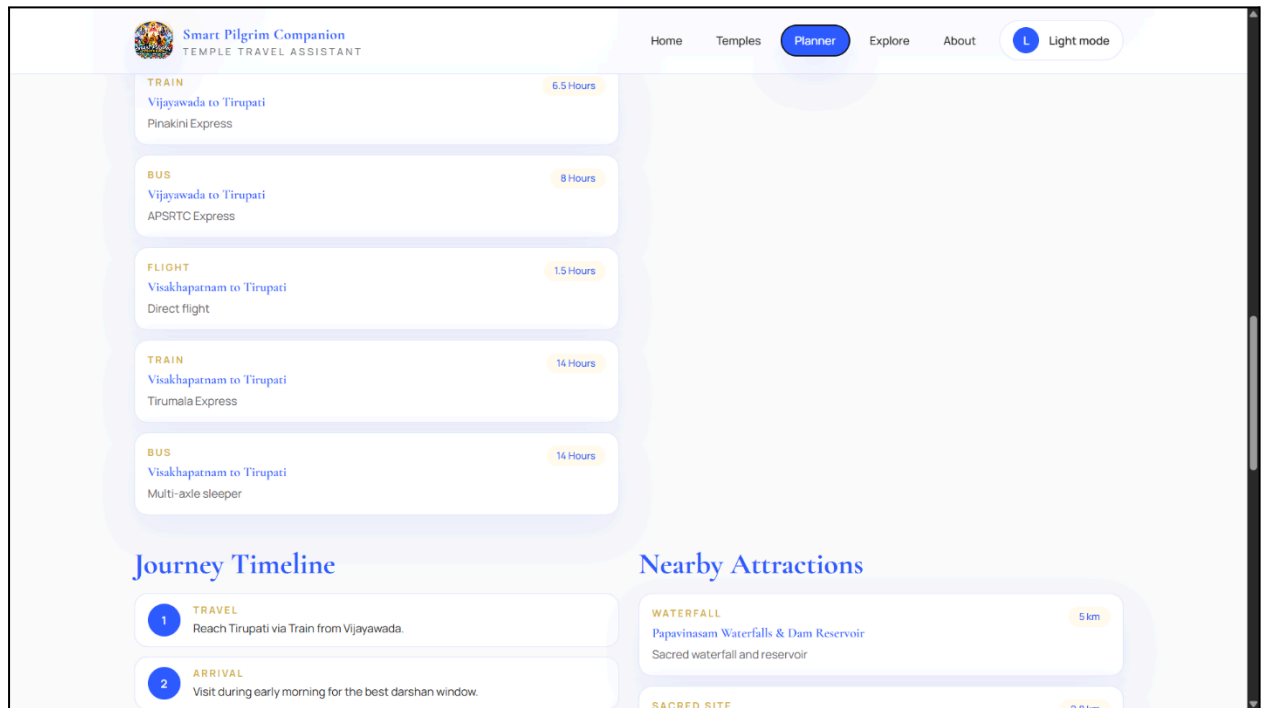


Figure 52: Final output screen 3

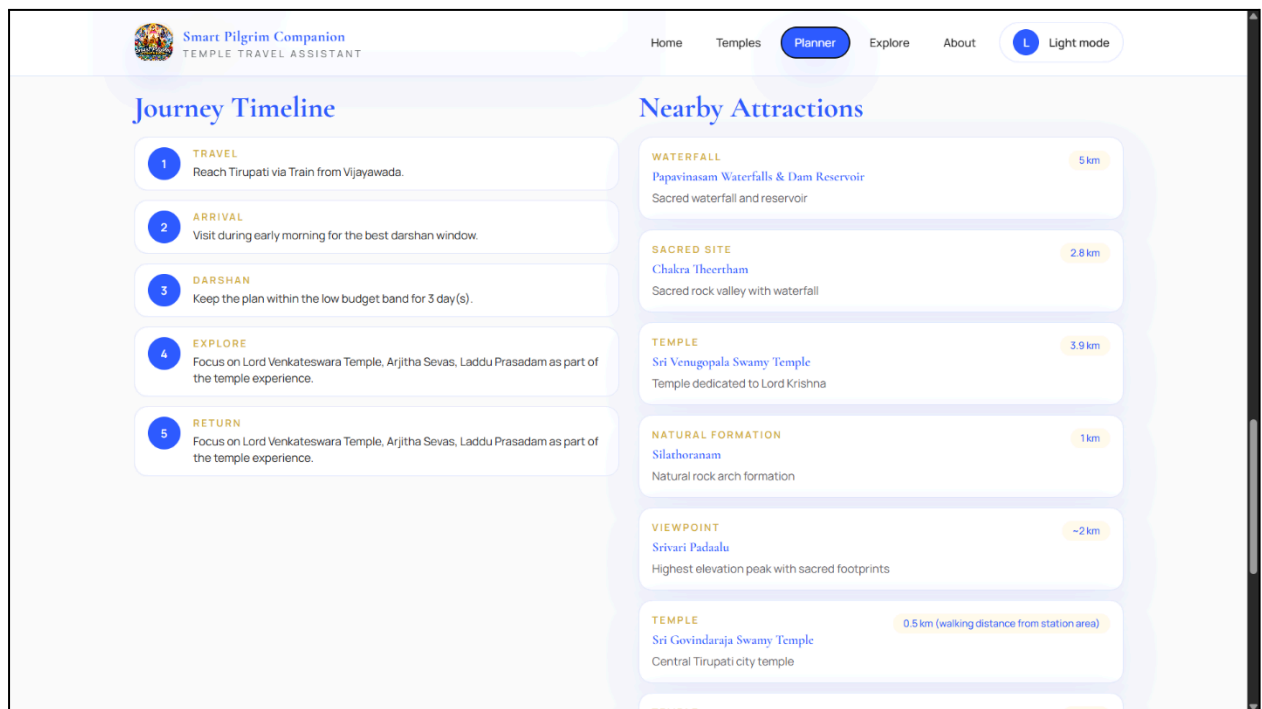


Figure 53: Final output screen 4

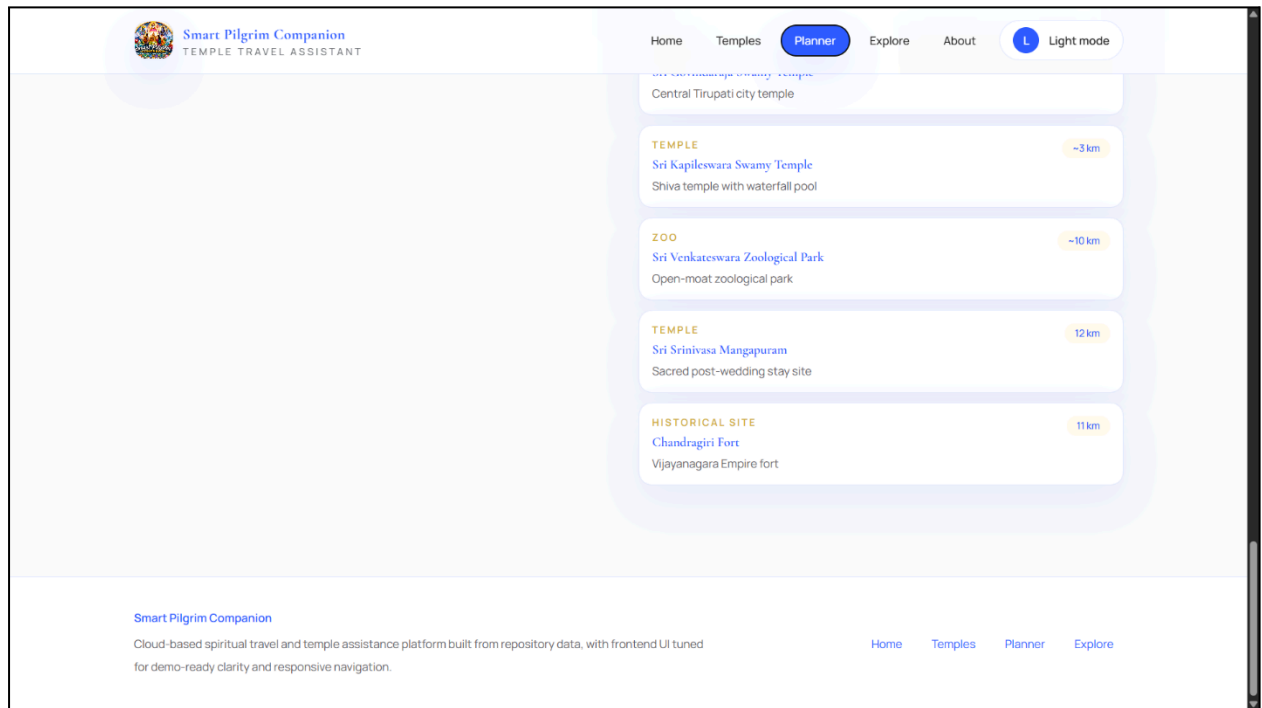


Figure 54: Final output screen 5

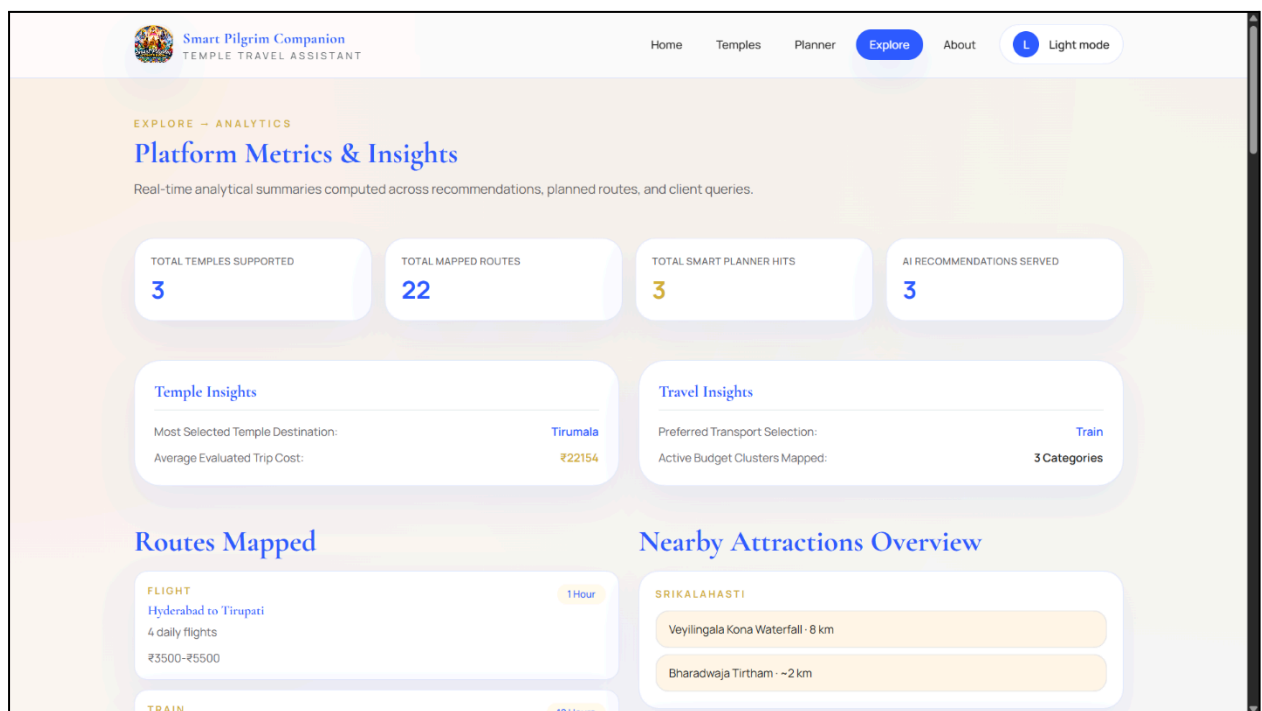


Figure 55: Final output screen 6

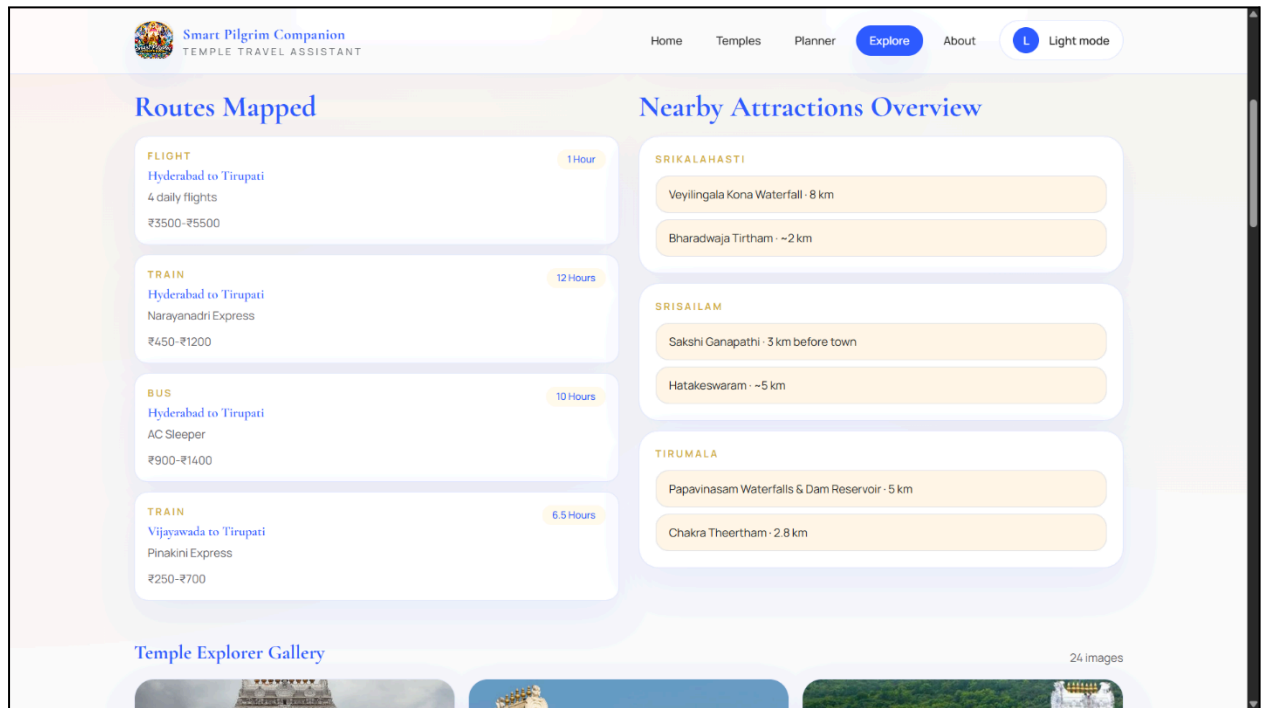


Figure 56: Final output screen 7

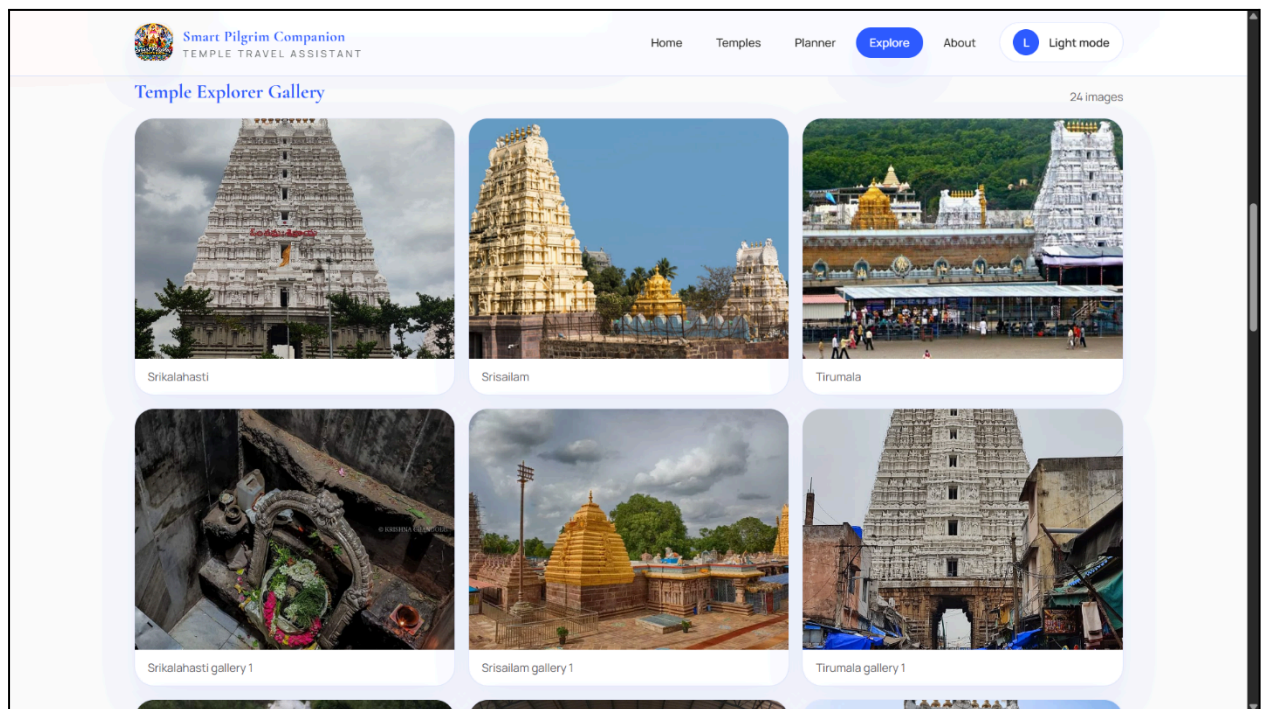


Figure 57: Final output screen 8

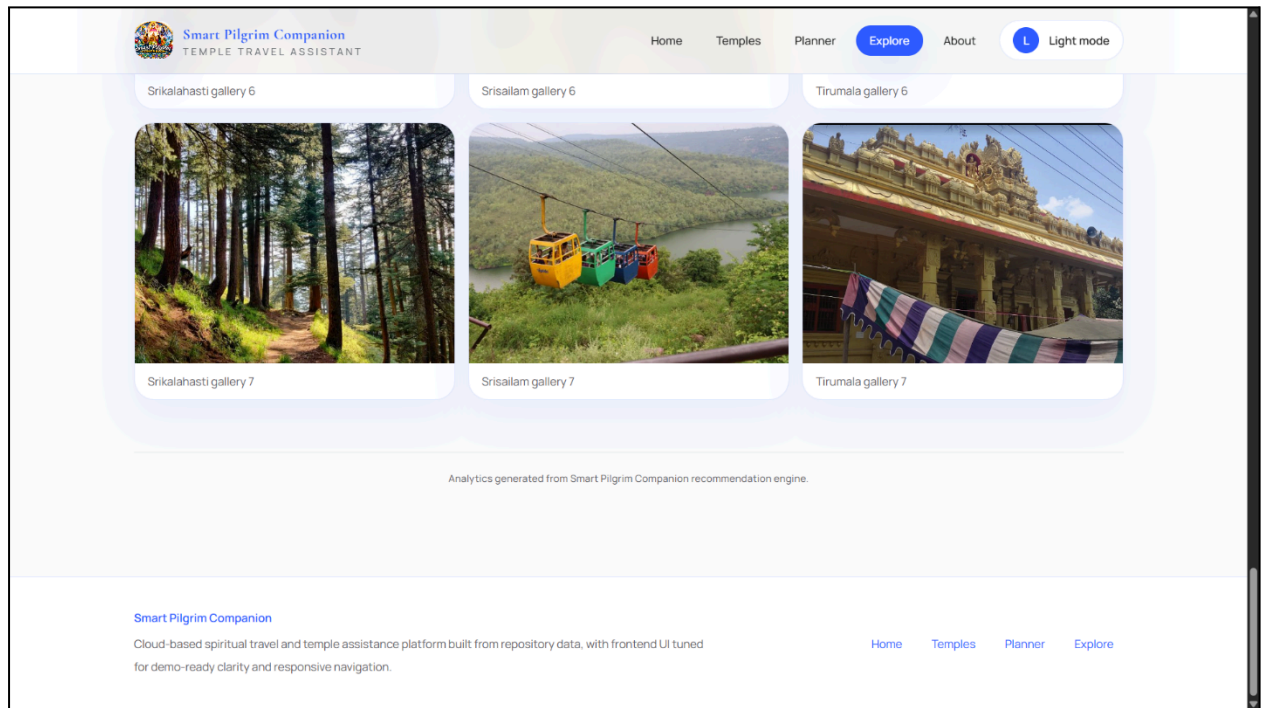


Figure 58: Final output screen 9

Chapter 9: Team Contributions

9.1 Team Contribution Overview

The Smart Pilgrim Companion project was completed through coordinated work across architecture, frontend validation, backend integration, database deployment, cloud monitoring, IAM, and documentation.

9.2 Contribution Mapping

Team Member	Contribution
Bhargav	Team Lead, Architecture, EC2 Deployment, Integration
Meghana	S3, UI Validation, Architecture Diagram
Soumya	RDS, Dataset Validation
Vishnu	Testing, CloudWatch Monitoring
Soumya2	IAM, Documentation

Table 5: Contribution Mapping

9.3 Responsibility Flow

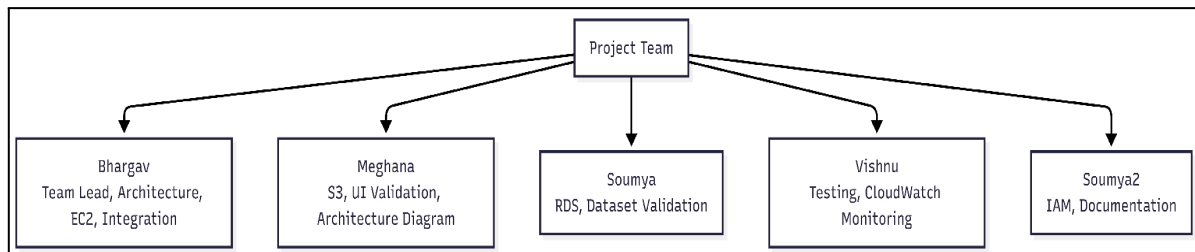


Figure 59: Responsibility Flow

9.4 Collaboration Evidence

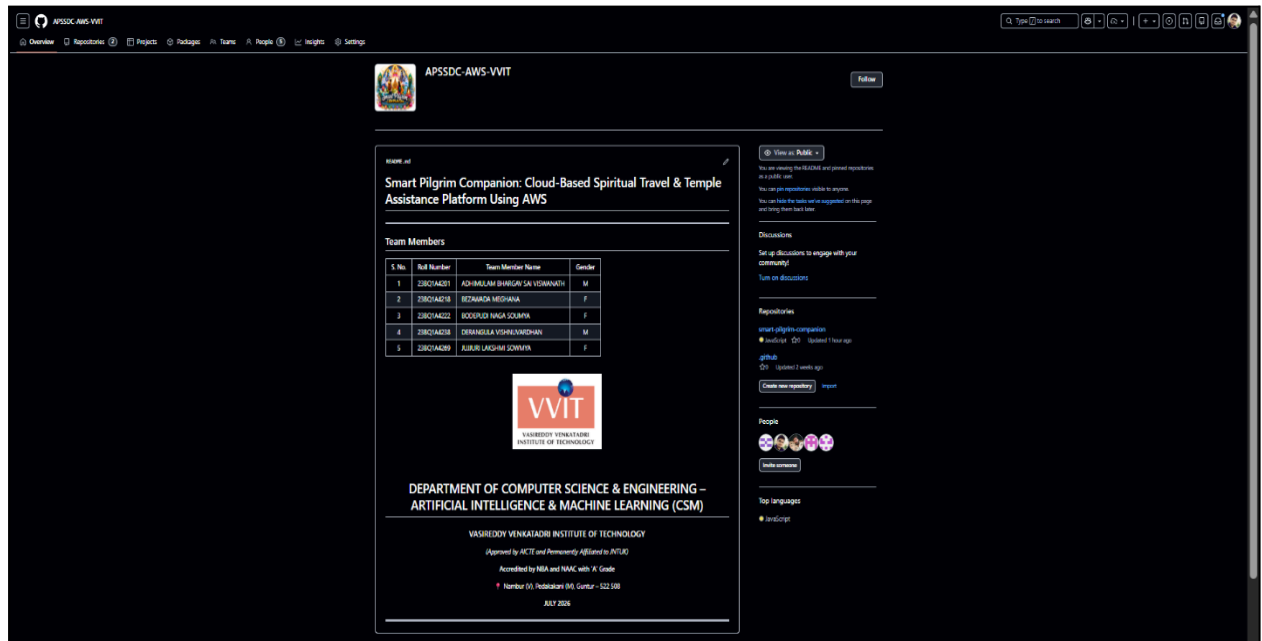


Figure 60: GitHub organization homepage 1

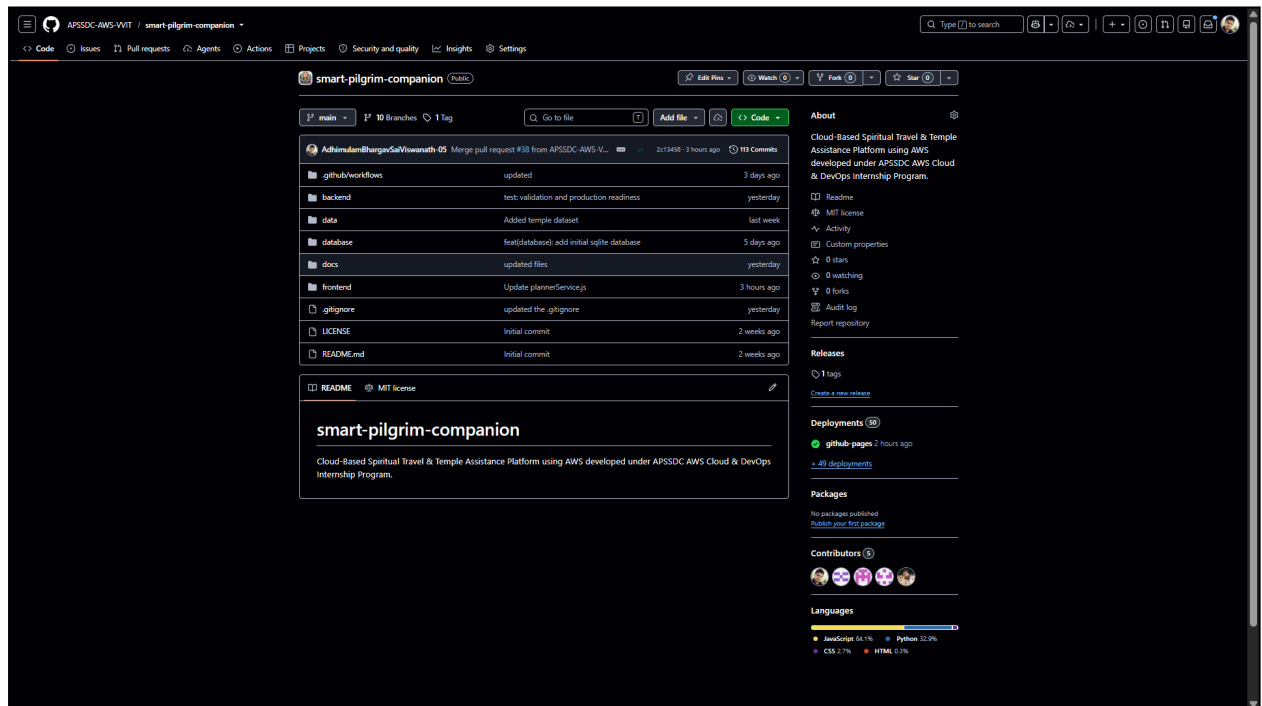


Figure 61: GitHub repository evidenc 1

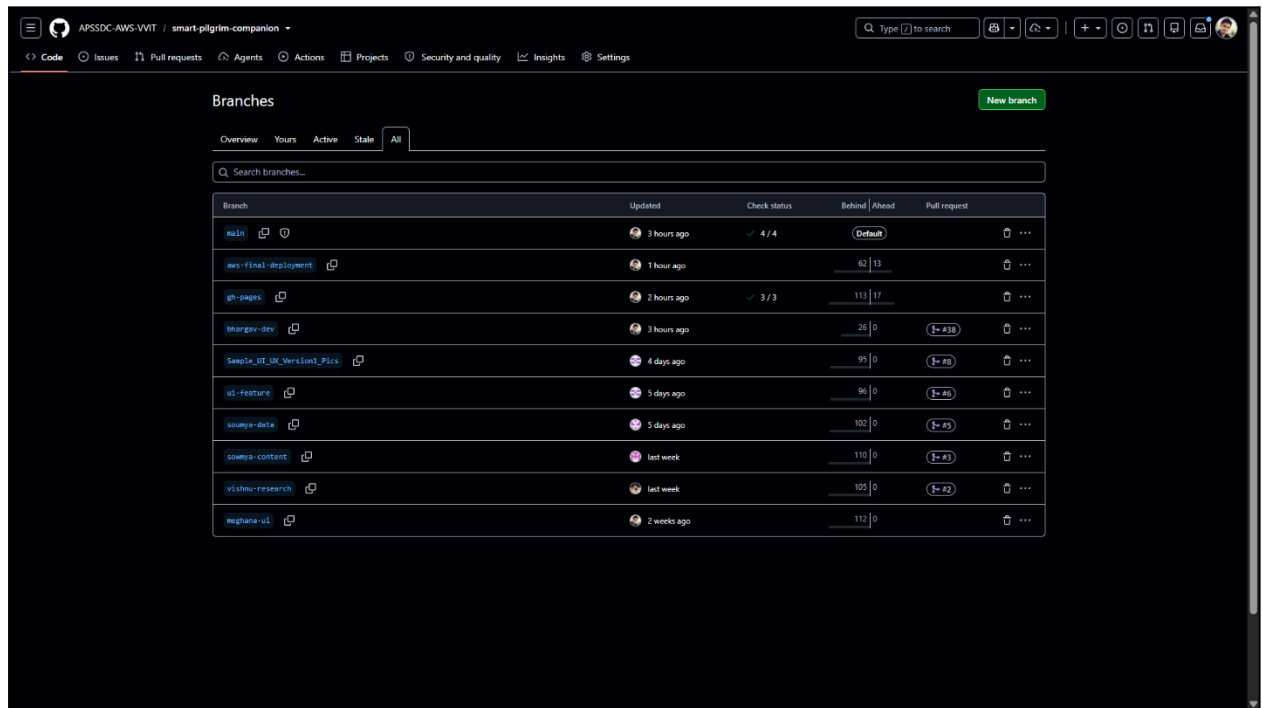


Figure 62 : GitHub branch collaboration e 1

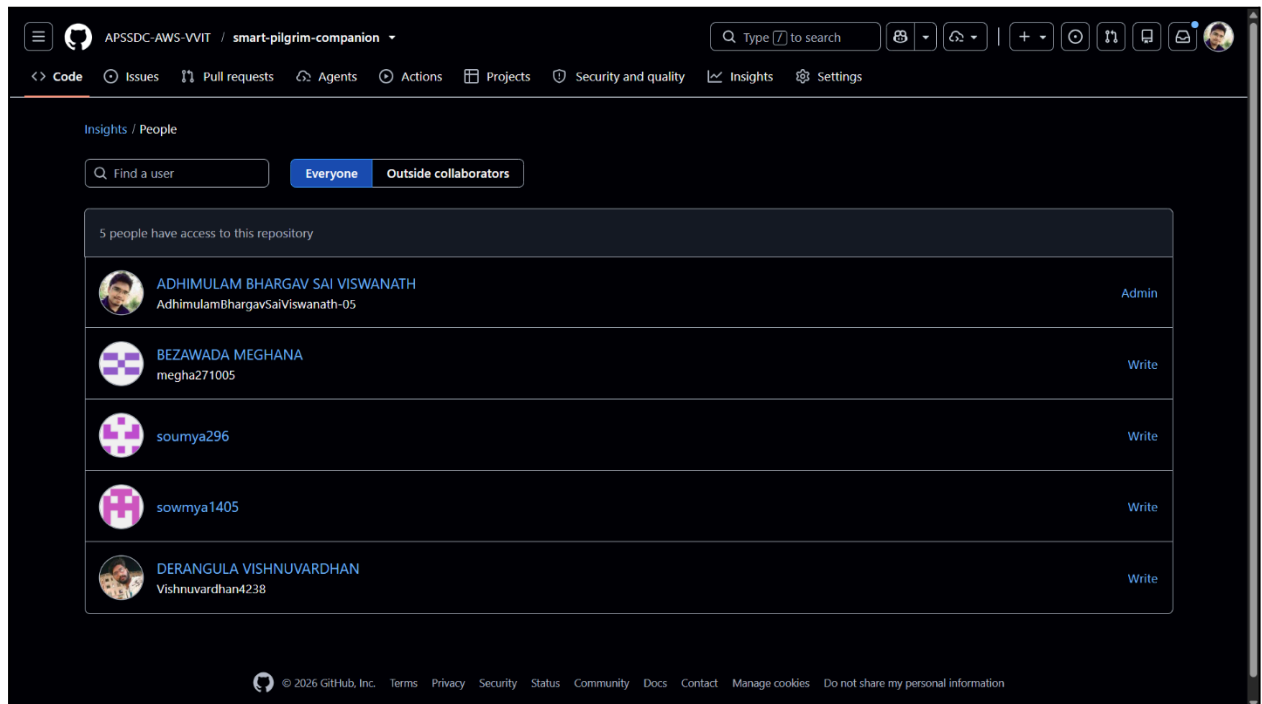


Figure 63 : Team members evidence 1

Chapter 10: Conclusion

Smart Pilgrim Companion successfully demonstrates a cloud-based spiritual travel and temple assistance platform using a real full-stack implementation.

The project includes a React + Vite frontend, Flask backend, structured datasets, SQLAlchemy models, local SQLite development support, GitHub Pages frontend deployment, and AWS deployment evidence using EC2, Nginx, Gunicorn, RDS MySQL, IAM, and CloudWatch.

The application supports temple exploration, temple details, route and budget planning, nearby place guidance, schedule information, and recommendation output. The backend APIs provide structured JSON responses, while the frontend service layer normalizes API data for user-facing pages.

The academic and deployment objectives were achieved through:

- Complete repository-based implementation.
- Localhost validation.
- GitHub deployment evidence.
- AWS migration evidence.
- RDS configuration and monitoring evidence.
- Nginx and Gunicorn service evidence.
- Testing and final output screenshots.
- Team contribution mapping.

The project is suitable for final academic submission because it combines practical cloud deployment, application development, database design, monitoring, testing, and documentation.

Chapter 11: Future Scope

11.1 Overview

The current project provides temple exploration, travel planning, budget guidance, route support, and AWS deployment evidence. Future work can extend the platform into a larger pilgrim assistance ecosystem.

11.2 Future Enhancements

- Add user authentication for saved travel plans.
- Add role-based admin screens for managing temple data.
- Add live darshan timing integration if official APIs become available.
- Add map-based route visualization.
- Add multi-language support for regional pilgrims.
- Add offline PDF itinerary export.
- Add notification support for reminders and travel alerts.
- Add richer analytics dashboards for popular temples, budget choices, and planner usage.
- Add automated CI/CD for backend deployment.
- Add automated database migration tooling for RDS schema updates.

11.3 Cloud Enhancement Scope

- Add HTTPS with SSL certificate management.
- Add load balancer support if traffic increases.
- Add private subnet architecture for database isolation.
- Add automated CloudWatch alarms.
- Add backup and restore strategy for RDS.
- Add infrastructure-as-code templates for repeatable deployment.

11.4 Future Scope Diagram

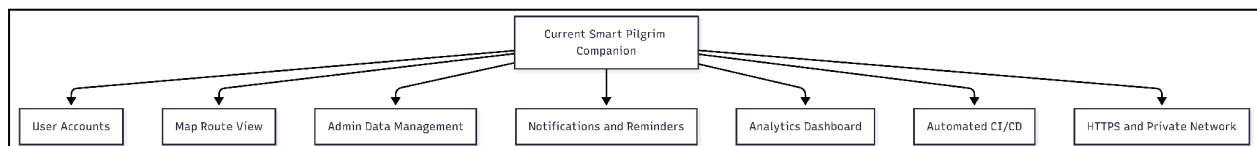


Figure 64 : Future Scope Diagram

References

Repository References

- README.md
- frontend/package.json
- frontend/DEPLOYMENT_GUIDE.md
- frontend/src/App.jsx
- frontend/src/services/api.js
- frontend/src/services/templeService.js
- frontend/src/services/plannerService.js
- backend/README.md
- backend/app.py
- backend/config.py
- backend/routes/temple_routes.py
- backend/services/planner_service.py
- backend/services/recommendation_engine.py
- database/schema.sql
- database/DATA_DICTIONARY.md
- deployment/aws/backend-audit.md
- deployment/aws/frontend-audit.md
- deployment/ProjectProofs/

Technology References

- [React documentation](#)
- [Vite documentation](#)
- [Flask documentation](#)
- [Flask-SQLAlchemy documentation](#)
- [Gunicorn documentation](#)
- [Nginx documentation](#)
- [AWS EC2 documentation](#)
- [AWS RDS documentation](#)
- [AWS IAM documentation](#)
- [AWS CloudWatch documentation](#)
- [GitHub Pages documentation](#)
- [Mermaid documentation](#)

Dataset References

- data/temples.csv
- data/travel_routes.csv

- data/budgets.csv
- data/schedules.csv
- data/temple_places.csv
- data/metadata.json
- data/user_scenarios.json

Evidence References

- deployment/ProjectProofs/development/
- deployment/ProjectProofs/github_deployment/
- deployment/ProjectProofs/aws_deployment/
- deployment/ProjectProofs/testing_evidence/
- deployment/ProjectProofs/team_contribution/
- deployment/ProjectProofs/final_output/